

A proof assistant-based formalisation of Core Erlang

Péter Bereczky, Dániel Horpácsi, Simon Thompson et al.

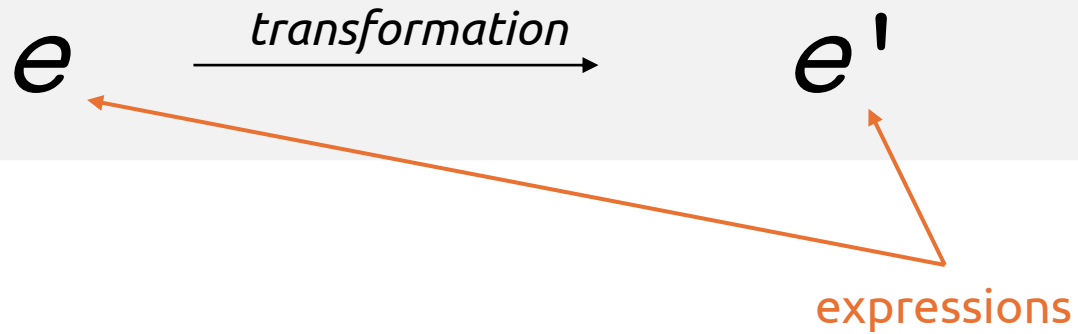
Alpine Verification Meeting 2026

Motivation: Correctness of refactoring

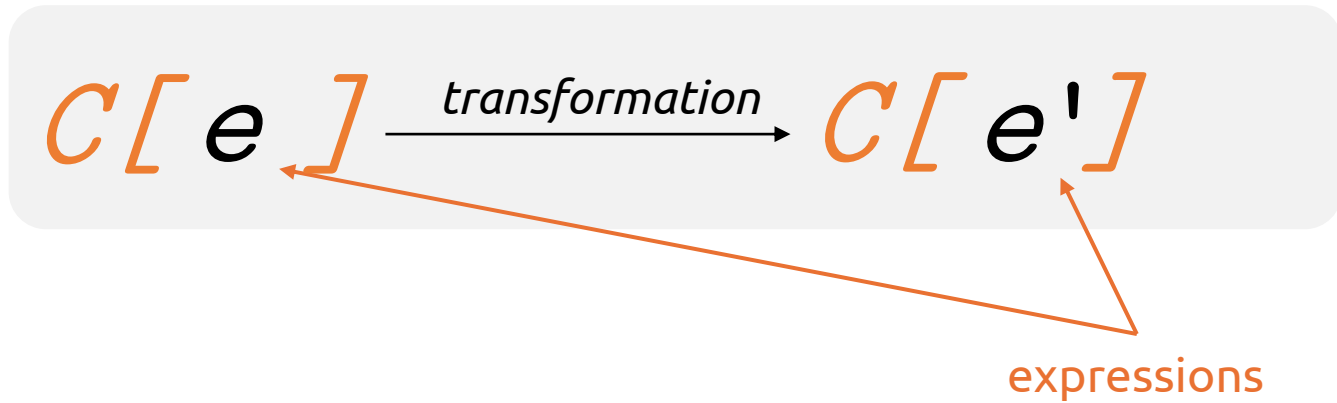
Motivation: Correctness of refactoring

$e \xrightarrow{\text{transformation}} e'$

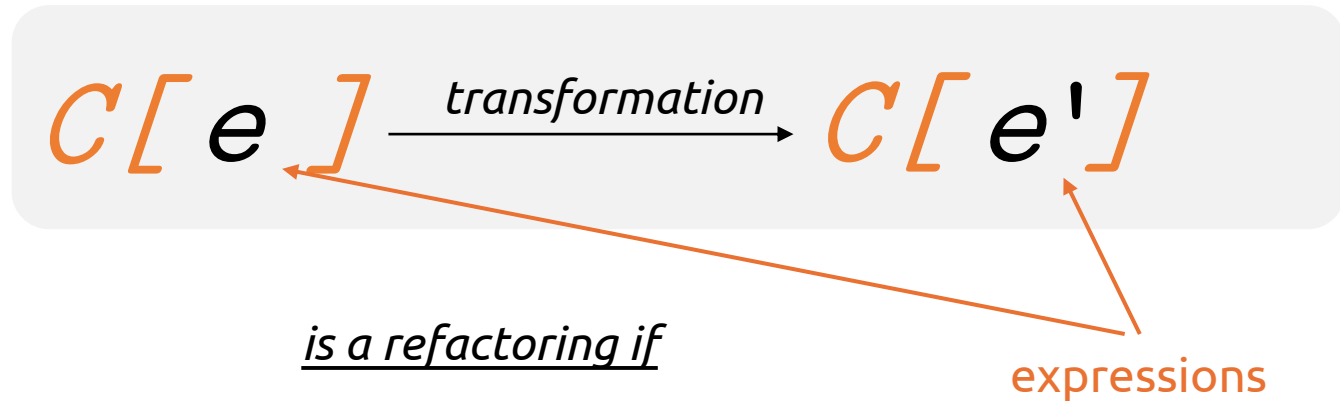
Motivation: Correctness of refactoring



Motivation: Correctness of refactoring



Motivation: Correctness of refactoring



Motivation: Correctness of refactoring

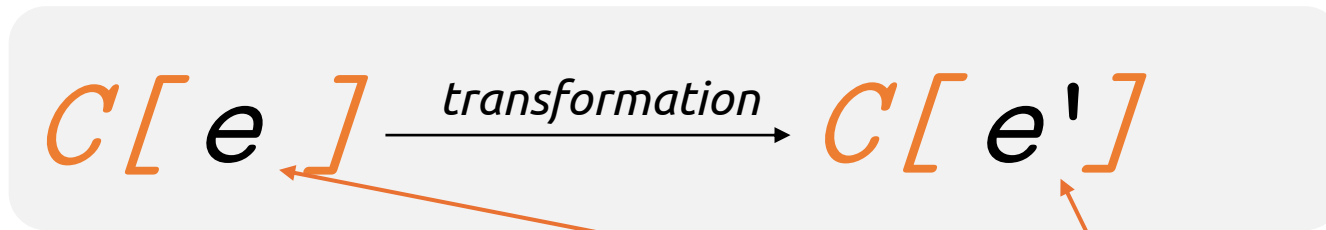
$C[e] \xrightarrow{\text{transformation}} C[e']$

is a refactoring if

expressions

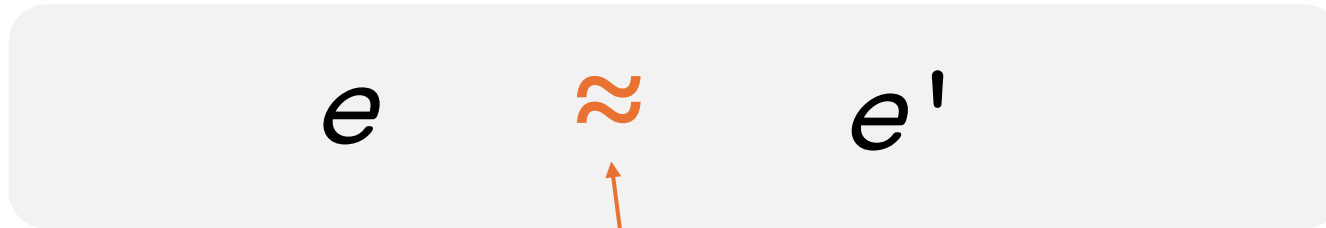
$e \approx e'$

Motivation: Correctness of refactoring



is a refactoring if

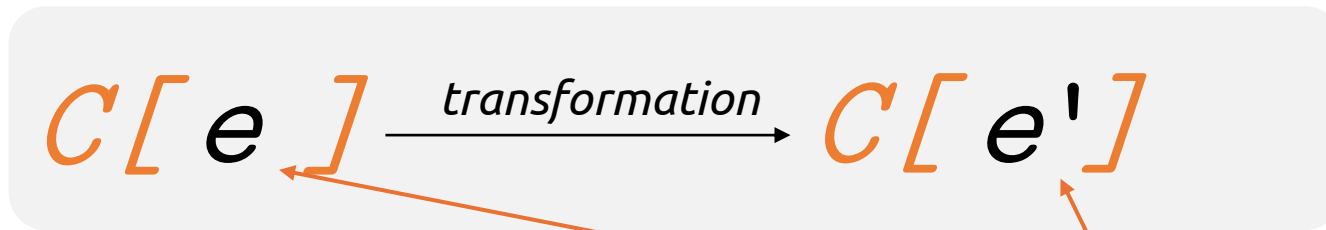
expressions



congruent program equivalence

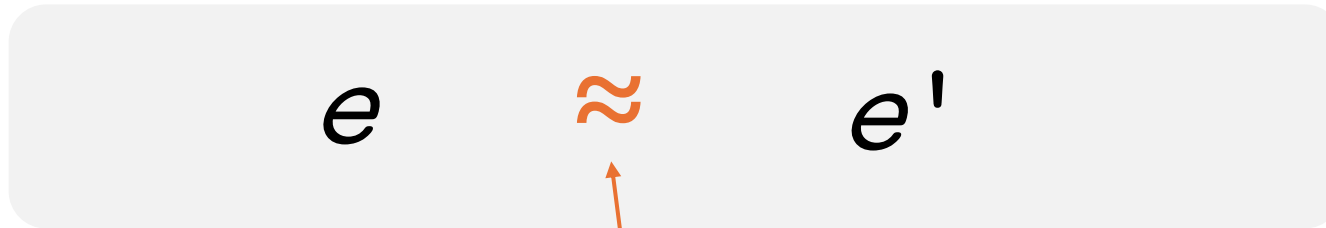
	static	dynamic
instance		
definition		

Motivation: Correctness of refactoring

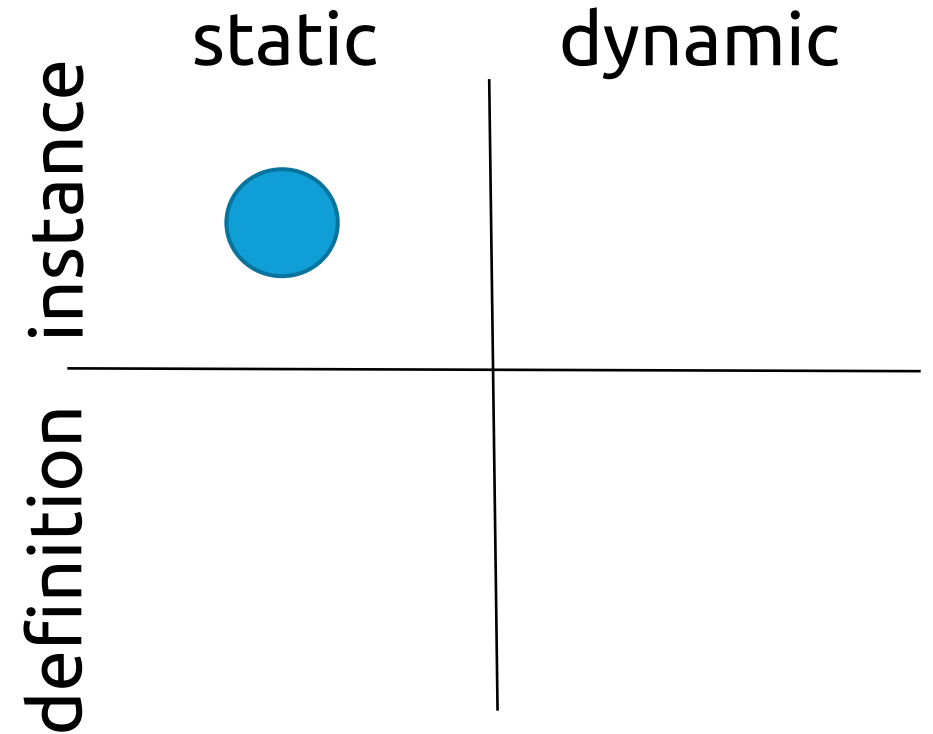


is a refactoring if

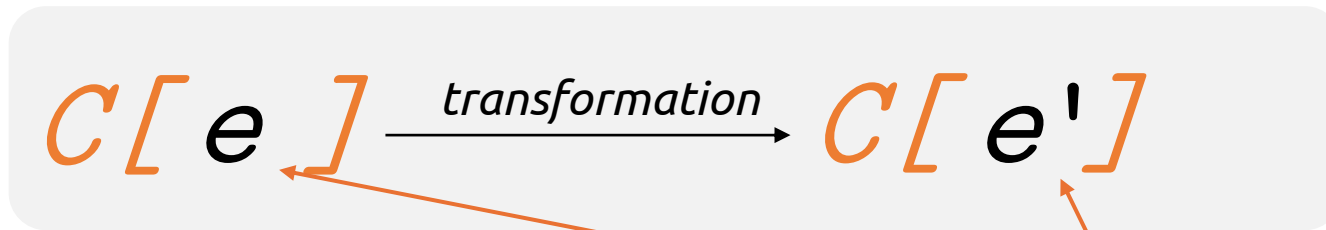
expressions



congruent program equivalence

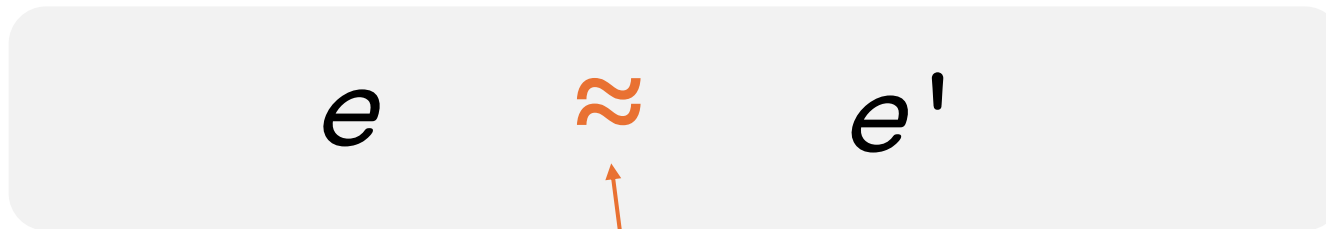


Motivation: Correctness of refactoring

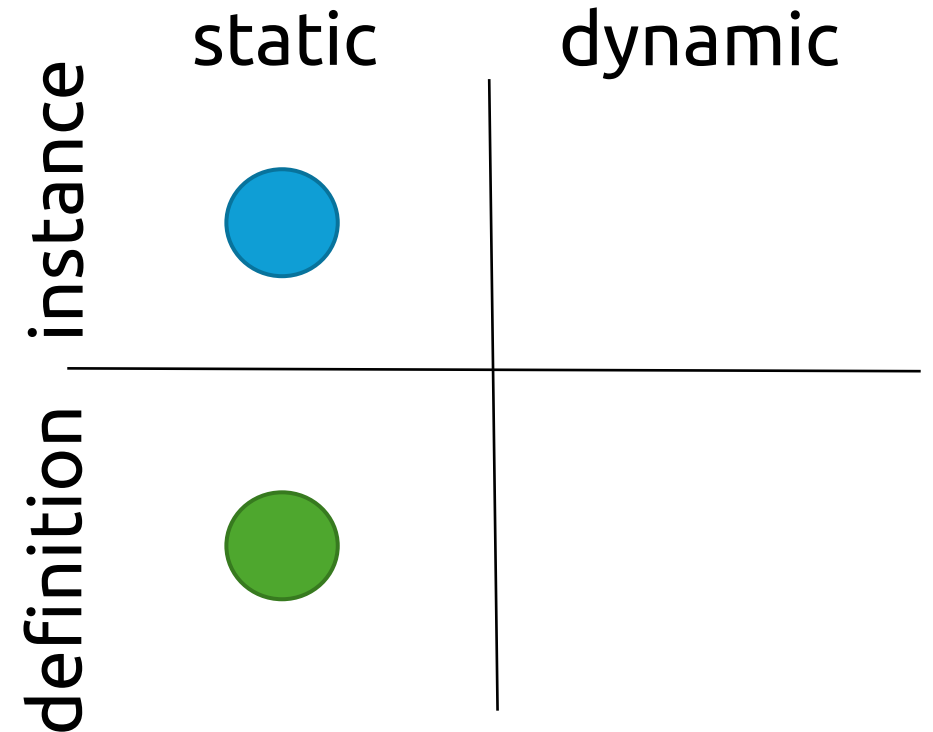


is a refactoring if

expressions



congruent program equivalence



The target language: Core Erlang

The target language: Core Erlang

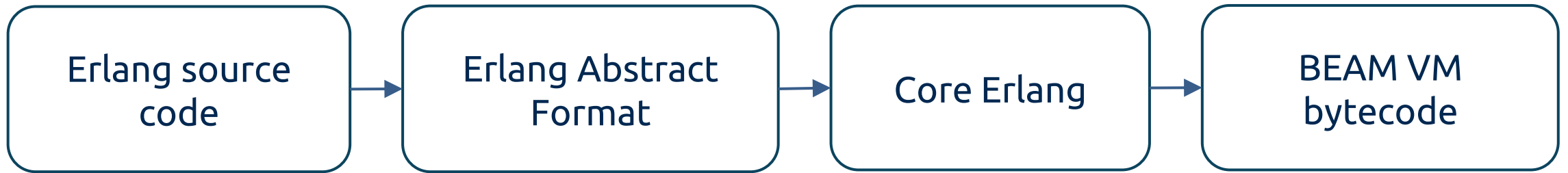
Erlang source
code



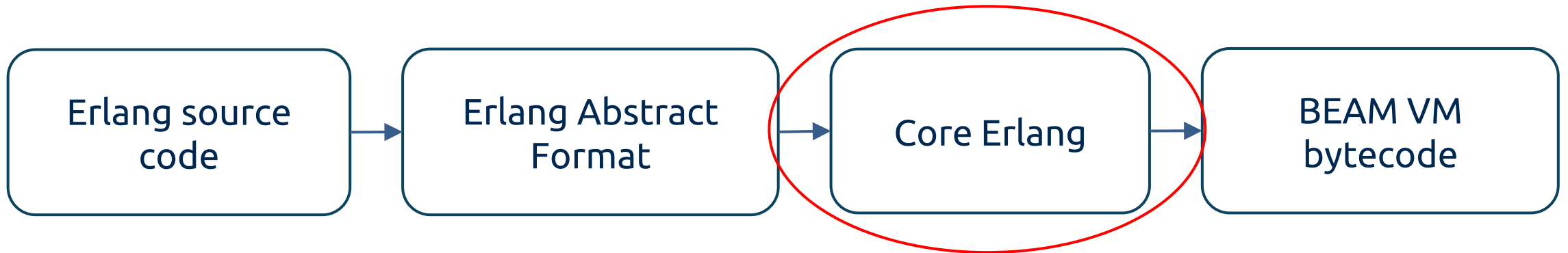
NOKIA



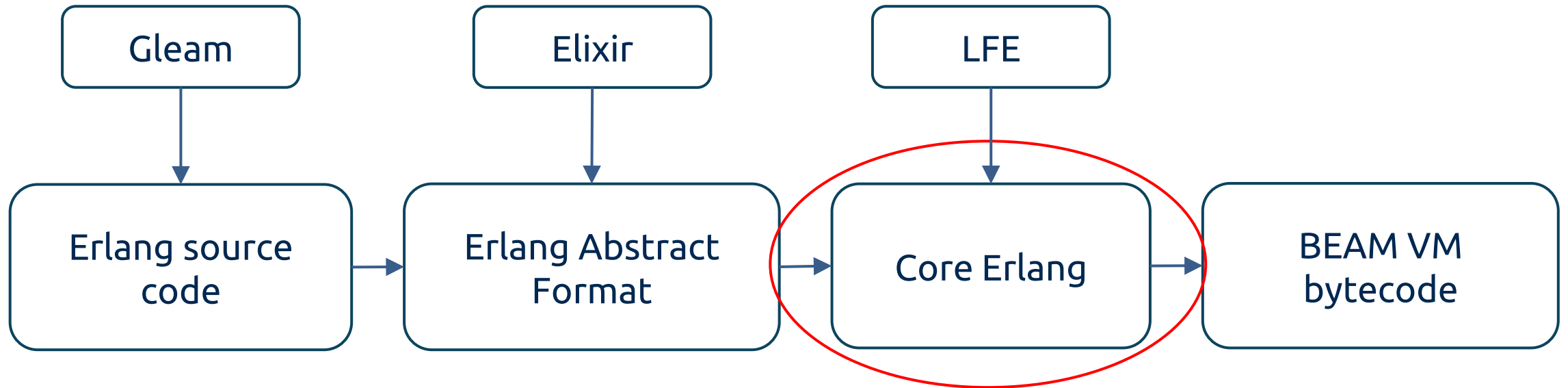
The target language: Core Erlang



The target language: Core Erlang



The target language: Core Erlang



Function closures

```
let X = 42 in
  let F = fun() -> X in
    let X = 0 in
      apply F()
```

Frame stack semantics

$\langle \emptyset, \text{let } X = 42 \text{ in let } F = \text{fun}() \rightarrow X \text{ in let } X = 0 \text{ in apply } F() \rangle \rightarrow$

Frame stack semantics

$$\langle K, \text{let } \langle x_1, \dots, x_n \rangle = e_1 \text{ in } e_2 \rangle \longrightarrow \langle \text{let } \langle x_1, \dots, x_n \rangle = \square \text{ in } e_2 :: K, e_1 \rangle$$
$$\langle \emptyset, \text{let } X = 42 \text{ in let } F = \text{fun}() \rightarrow X \text{ in let } X = 0 \text{ in apply } F() \rangle \rightarrow$$

Frame stack semantics

$$\langle K, \text{let } \langle x_1, \dots, x_n \rangle = e_1 \text{ in } e_2 \rangle \longrightarrow \langle \text{let } \langle x_1, \dots, x_n \rangle = \square \text{ in } e_2 :: K, e_1 \rangle$$
$$\langle \emptyset, \text{let } X = 42 \text{ in let } F = \text{fun}() \rightarrow X \text{ in let } X = 0 \text{ in apply } F() \rangle \rightarrow$$
$$\langle \text{let } X = \blacksquare \text{ in let } F = \text{fun}() \rightarrow X \text{ in let } X = 0 \text{ in apply } F() :: \emptyset, 42 \rangle \rightarrow$$

Frame stack semantics

$$\langle \text{let } \langle x_1, \dots, x_n \rangle = \square \text{ in } e_2 :: K, \langle v_1, \dots, v_n \rangle \rangle \longrightarrow \langle K, e_2[x_1 \mapsto v_1, \dots, x_n \mapsto v_n] \rangle$$

$\langle \emptyset, \text{let } X = 42 \text{ in let } F = \text{fun}() \rightarrow X \text{ in let } X = 0 \text{ in apply } F() \rangle \rightarrow$

$\langle \text{let } X = \blacksquare \text{ in let } F = \text{fun}() \rightarrow X \text{ in let } X = 0 \text{ in apply } F() :: \emptyset, 42 \rangle \rightarrow$

Frame stack semantics

$$\langle \text{let } \langle x_1, \dots, x_n \rangle = \square \text{ in } e_2 :: K, \langle v_1, \dots, v_n \rangle \rangle \longrightarrow \langle K, e_2[x_1 \mapsto v_1, \dots, x_n \mapsto v_n] \rangle$$

$\langle \emptyset, \text{let } X = 42 \text{ in let } F = \text{fun}() \rightarrow X \text{ in let } X = 0 \text{ in apply } F() \rangle \rightarrow$

$\langle \text{let } X = \blacksquare \text{ in let } F = \text{fun}() \rightarrow X \text{ in let } X = 0 \text{ in apply } F() :: \emptyset, 42 \rangle \rightarrow$

$\langle \emptyset, \text{let } F = \text{fun}() \rightarrow 42 \text{ in let } X = 0 \text{ in apply } F() \rangle \rightarrow$

Frame stack semantics

$\langle \emptyset, \text{let } X = 42 \text{ in let } F = \text{fun}() \rightarrow X \text{ in let } X = 0 \text{ in apply } F() \rangle \rightarrow$
 $\langle \text{let } X = \blacksquare \text{ in let } F = \text{fun}() \rightarrow X \text{ in let } X = 0 \text{ in apply } F() :: \emptyset, 42 \rangle \rightarrow$
 $\langle \emptyset, \text{let } F = \text{fun}() \rightarrow 42 \text{ in let } X = 0 \text{ in apply } F() \rangle \rightarrow$
 $\langle \text{let } F = \blacksquare \text{ in let } X = 0 \text{ in apply } F() :: \emptyset, \text{fun}() \rightarrow 42 \rangle \rightarrow$

Frame stack semantics

$\langle \emptyset, \text{let } X = 42 \text{ in let } F = \text{fun}() \rightarrow X \text{ in let } X = 0 \text{ in apply } F() \rangle \rightarrow$
 $\langle \text{let } X = \blacksquare \text{ in let } F = \text{fun}() \rightarrow X \text{ in let } X = 0 \text{ in apply } F() :: \emptyset, 42 \rangle \rightarrow$
 $\langle \emptyset, \text{let } F = \text{fun}() \rightarrow 42 \text{ in let } X = 0 \text{ in apply } F() \rangle \rightarrow$
 $\langle \text{let } F = \blacksquare \text{ in let } X = 0 \text{ in apply } F() :: \emptyset, \text{fun}() \rightarrow 42 \rangle \rightarrow$
 $\langle \text{let } F = \blacksquare \text{ in let } X = 0 \text{ in apply } F() :: \emptyset, \text{clos}([], 42) \rangle \rightarrow$

Frame stack semantics

$\langle \emptyset, \text{let } X = 42 \text{ in let } F = \text{fun}() \rightarrow X \text{ in let } X = 0 \text{ in apply } F() \rangle \rightarrow$
 $\langle \text{let } X = \blacksquare \text{ in let } F = \text{fun}() \rightarrow X \text{ in let } X = 0 \text{ in apply } F() :: \emptyset, 42 \rangle \rightarrow$
 $\langle \emptyset, \text{let } F = \text{fun}() \rightarrow 42 \text{ in let } X = 0 \text{ in apply } F() \rangle \rightarrow$
 $\langle \text{let } F = \blacksquare \text{ in let } X = 0 \text{ in apply } F() :: \emptyset, \text{fun}() \rightarrow 42 \rangle \rightarrow$
 $\langle \text{let } F = \blacksquare \text{ in let } X = 0 \text{ in apply } F() :: \emptyset, \text{clos}([], 42) \rangle \rightarrow$
 $\langle \emptyset, \text{let } X = 0 \text{ in apply } \text{clos}([], 42)() \rangle \rightarrow$

Frame stack semantics

$\langle \emptyset, \text{let } X = 42 \text{ in let } F = \text{fun}() \rightarrow X \text{ in let } X = 0 \text{ in apply } F() \rangle \rightarrow$
 $\langle \text{let } X = \blacksquare \text{ in let } F = \text{fun}() \rightarrow X \text{ in let } X = 0 \text{ in apply } F() :: \emptyset, 42 \rangle \rightarrow$
 $\langle \emptyset, \text{let } F = \text{fun}() \rightarrow 42 \text{ in let } X = 0 \text{ in apply } F() \rangle \rightarrow$
 $\langle \text{let } F = \blacksquare \text{ in let } X = 0 \text{ in apply } F() :: \emptyset, \text{fun}() \rightarrow 42 \rangle \rightarrow$
 $\langle \text{let } F = \blacksquare \text{ in let } X = 0 \text{ in apply } F() :: \emptyset, \text{clos}([], 42) \rangle \rightarrow$
 $\langle \emptyset, \text{let } X = 0 \text{ in apply } \text{clos}([], 42)() \rangle \rightarrow$
 $\langle \text{let } X = \blacksquare \text{ in apply } \text{clos}([], 42)() :: \emptyset, 0 \rangle \rightarrow$

Frame stack semantics

$\langle \emptyset, \text{let } X = 42 \text{ in let } F = \text{fun}() \rightarrow X \text{ in let } X = 0 \text{ in apply } F() \rangle \rightarrow$
 $\langle \text{let } X = \blacksquare \text{ in let } F = \text{fun}() \rightarrow X \text{ in let } X = 0 \text{ in apply } F() :: \emptyset, 42 \rangle \rightarrow$
 $\langle \emptyset, \text{let } F = \text{fun}() \rightarrow 42 \text{ in let } X = 0 \text{ in apply } F() \rangle \rightarrow$
 $\langle \text{let } F = \blacksquare \text{ in let } X = 0 \text{ in apply } F() :: \emptyset, \text{fun}() \rightarrow 42 \rangle \rightarrow$
 $\langle \text{let } F = \blacksquare \text{ in let } X = 0 \text{ in apply } F() :: \emptyset, \text{clos}([], 42) \rangle \rightarrow$
 $\langle \emptyset, \text{let } X = 0 \text{ in apply } \text{clos}([], 42)() \rangle \rightarrow$
 $\langle \text{let } X = \blacksquare \text{ in apply } \text{clos}([], 42)() :: \emptyset, 0 \rangle \rightarrow$
 $\langle \emptyset, \text{apply } \text{clos}([], 42)() \rangle \rightarrow$

Frame stack semantics

$\langle \emptyset, \text{let } X = 42 \text{ in let } F = \text{fun}() \rightarrow X \text{ in let } X = 0 \text{ in apply } F() \rangle \rightarrow$
 $\langle \text{let } X = \blacksquare \text{ in let } F = \text{fun}() \rightarrow X \text{ in let } X = 0 \text{ in apply } F() :: \emptyset, 42 \rangle \rightarrow$
 $\langle \emptyset, \text{let } F = \text{fun}() \rightarrow 42 \text{ in let } X = 0 \text{ in apply } F() \rangle \rightarrow$
 $\langle \text{let } F = \blacksquare \text{ in let } X = 0 \text{ in apply } F() :: \emptyset, \text{fun}() \rightarrow 42 \rangle \rightarrow$
 $\langle \text{let } F = \blacksquare \text{ in let } X = 0 \text{ in apply } F() :: \emptyset, \text{clos}([], 42) \rangle \rightarrow$
 $\langle \emptyset, \text{let } X = 0 \text{ in apply } \text{clos}([], 42)() \rangle \rightarrow$
 $\langle \text{let } X = \blacksquare \text{ in apply } \text{clos}([], 42)() :: \emptyset, 0 \rangle \rightarrow$
 $\langle \emptyset, \text{apply } \text{clos}([], 42)() \rangle \rightarrow$
 $\langle \text{apply } \blacksquare() :: \emptyset, \text{clos}([], 42) \rangle \rightarrow$

Frame stack semantics

$\langle \emptyset, \text{let } X = 42 \text{ in let } F = \text{fun}() \rightarrow X \text{ in let } X = 0 \text{ in apply } F() \rangle \rightarrow$
 $\langle \text{let } X = \blacksquare \text{ in let } F = \text{fun}() \rightarrow X \text{ in let } X = 0 \text{ in apply } F() :: \emptyset, 42 \rangle \rightarrow$
 $\langle \emptyset, \text{let } F = \text{fun}() \rightarrow 42 \text{ in let } X = 0 \text{ in apply } F() \rangle \rightarrow$
 $\langle \text{let } F = \blacksquare \text{ in let } X = 0 \text{ in apply } F() :: \emptyset, \text{fun}() \rightarrow 42 \rangle \rightarrow$
 $\langle \text{let } F = \blacksquare \text{ in let } X = 0 \text{ in apply } F() :: \emptyset, \text{clos}([], 42) \rangle \rightarrow$
 $\langle \emptyset, \text{let } X = 0 \text{ in apply } \text{clos}([], 42)() \rangle \rightarrow$
 $\langle \text{let } X = \blacksquare \text{ in apply } \text{clos}([], 42)() :: \emptyset, 0 \rangle \rightarrow$
 $\langle \emptyset, \text{apply } \text{clos}([], 42)() \rangle \rightarrow$
 $\langle \text{apply } \blacksquare() :: \emptyset, \text{clos}([], 42) \rangle \rightarrow$
 $\langle \emptyset, 42 \rangle$

Modular semantics

Level of the semantics	Formal relation
Sequential semantics	$\langle K, e \rangle \rightarrow \langle K', e' \rangle$

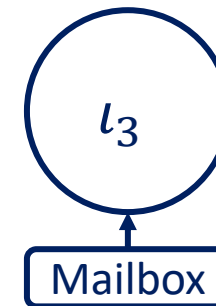
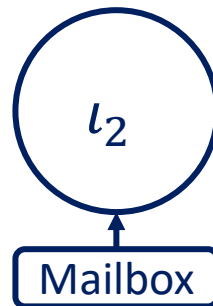
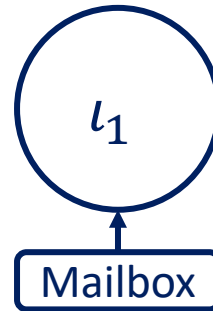
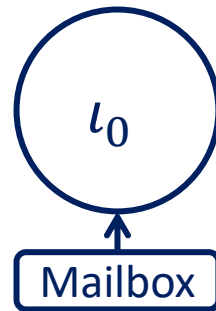
Modular semantics

Level of the semantics

Formal relation

Sequential semantics

$\langle K, e \rangle \rightarrow \langle K', e' \rangle$



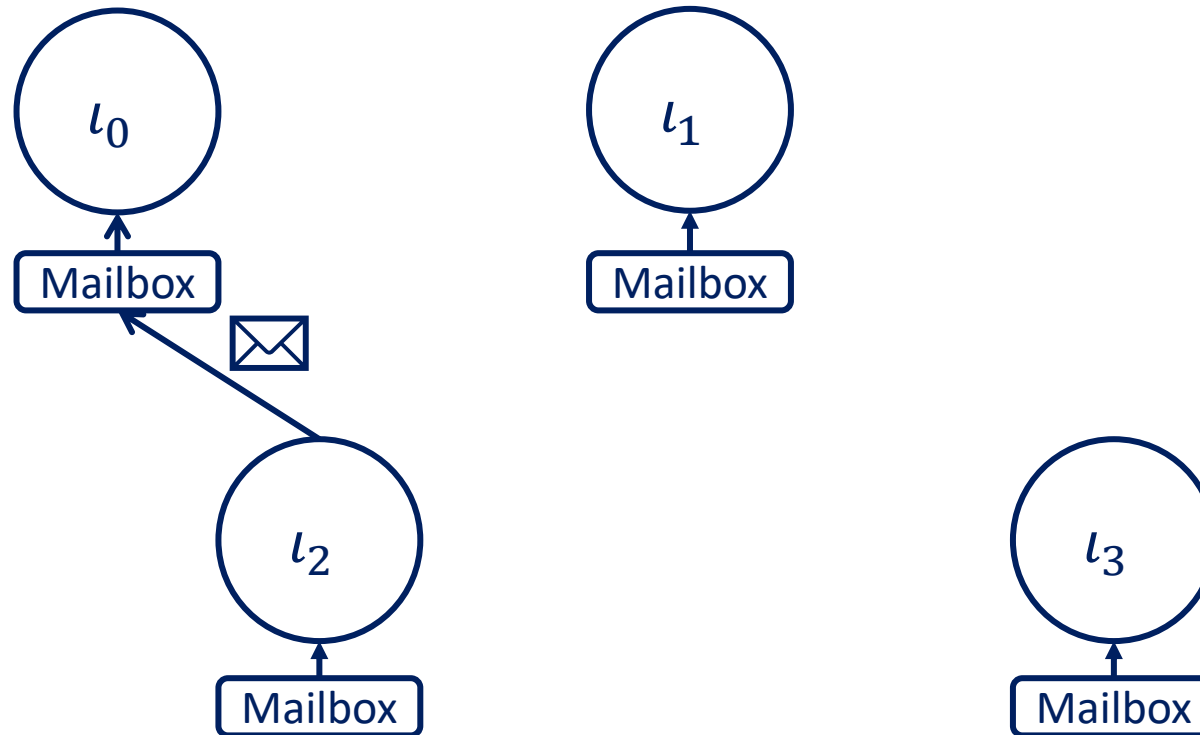
Modular semantics

Level of the semantics

Formal relation

Sequential semantics

$\langle K, e \rangle \rightarrow \langle K', e' \rangle$



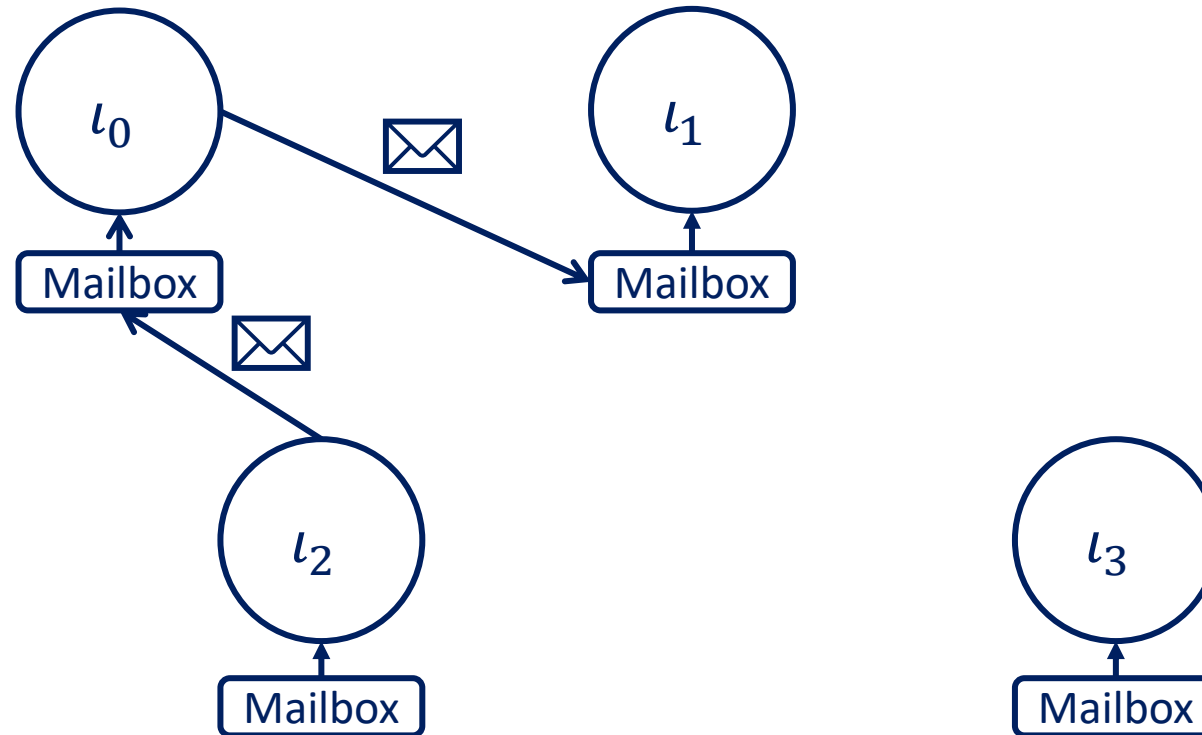
Modular semantics

Level of the semantics

Formal relation

Sequential semantics

$\langle K, e \rangle \rightarrow \langle K', e' \rangle$



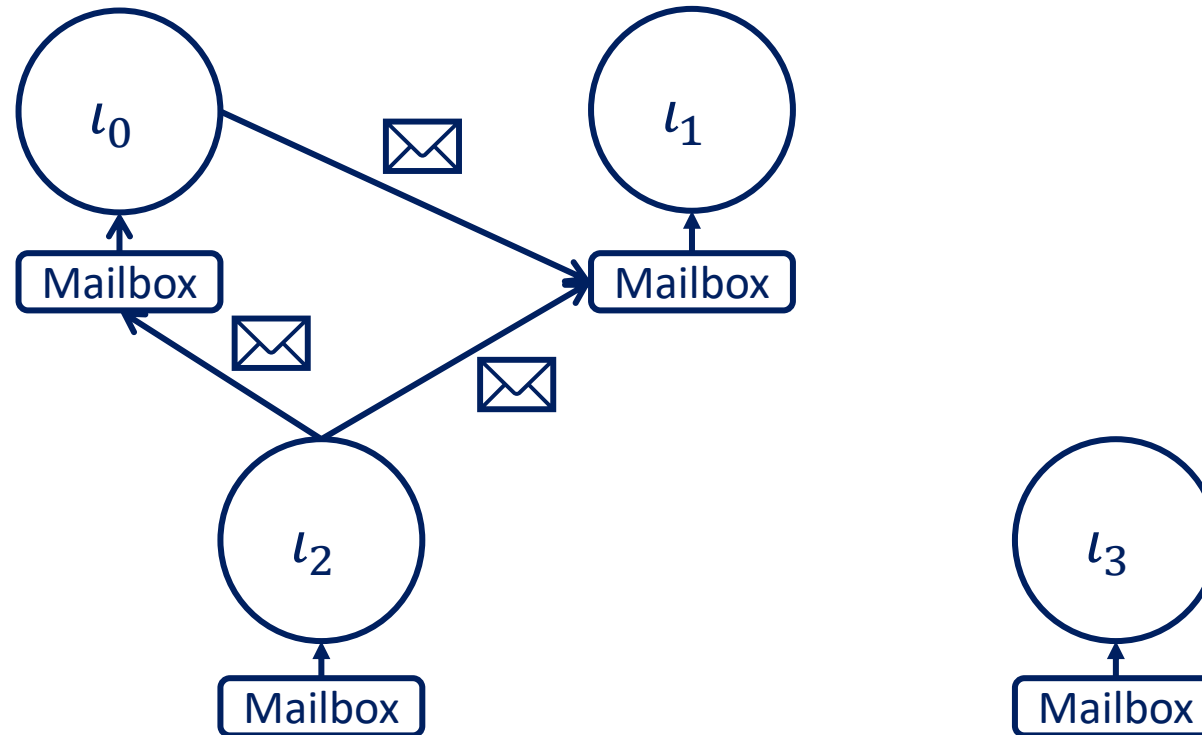
Modular semantics

Level of the semantics

Formal relation

Sequential semantics

$\langle K, e \rangle \rightarrow \langle K', e' \rangle$



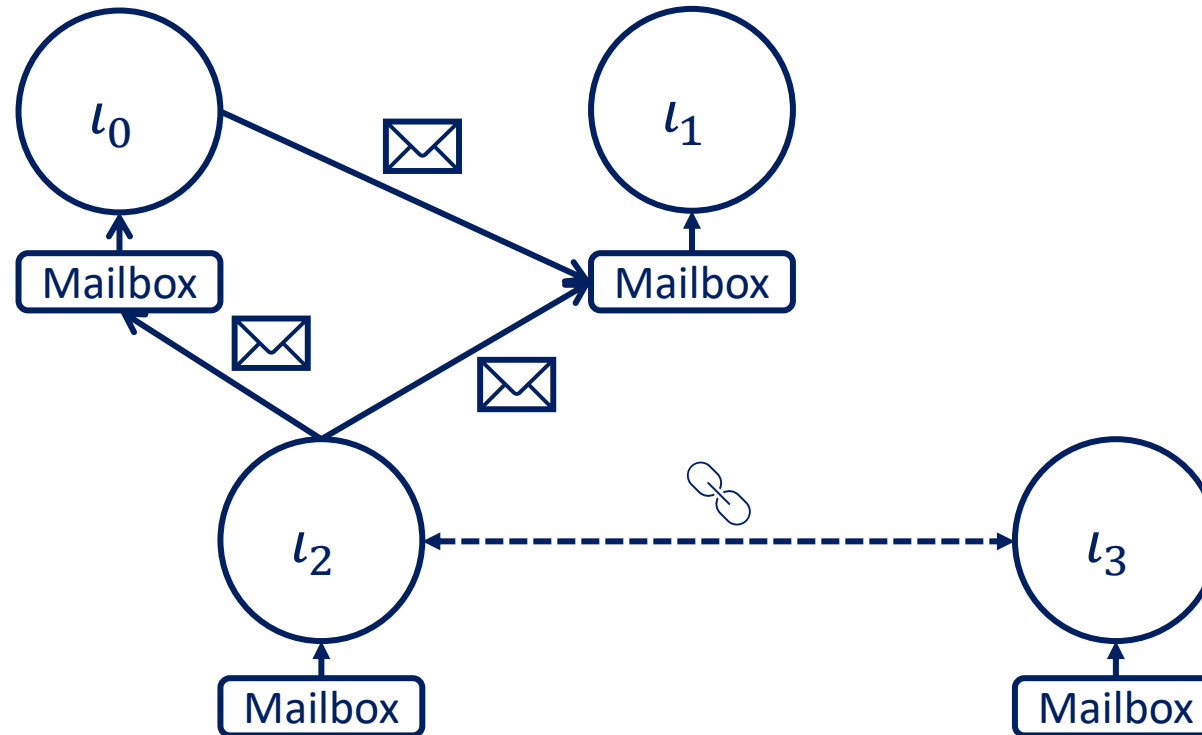
Modular semantics

Level of the semantics

Formal relation

Sequential semantics

$\langle K, e \rangle \rightarrow \langle K', e' \rangle$



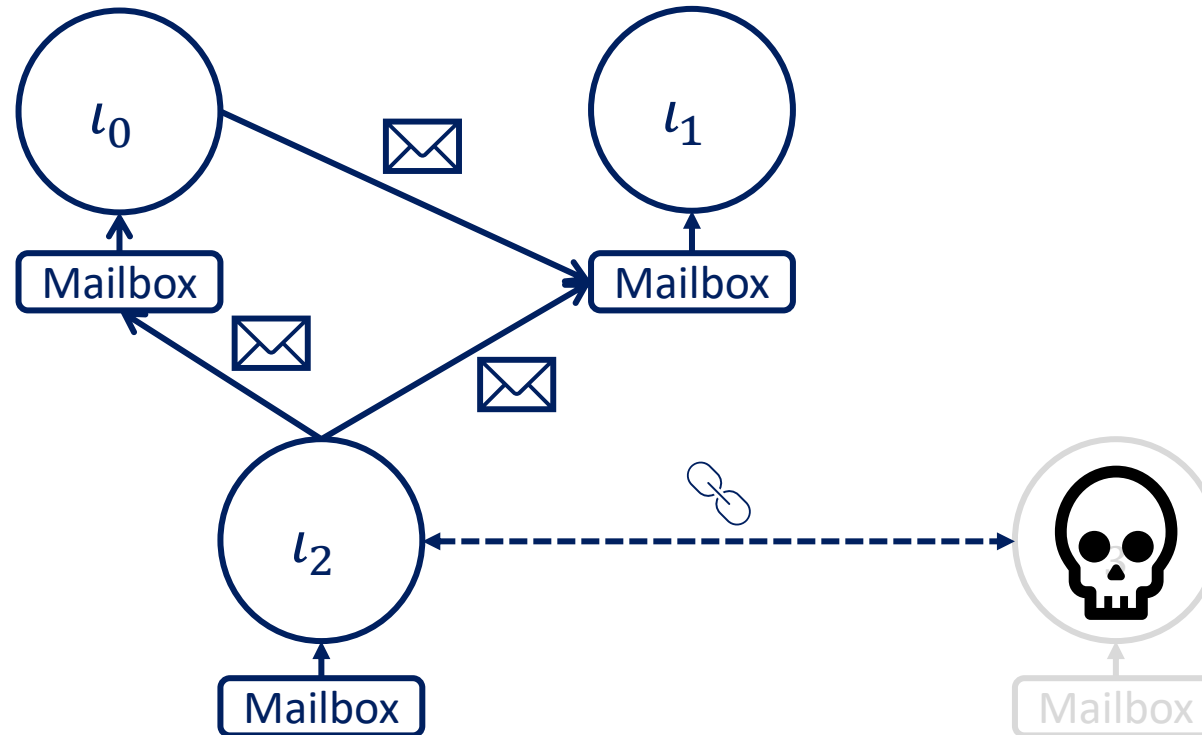
Modular semantics

Level of the semantics

Formal relation

Sequential semantics

$\langle K, e \rangle \rightarrow \langle K', e' \rangle$



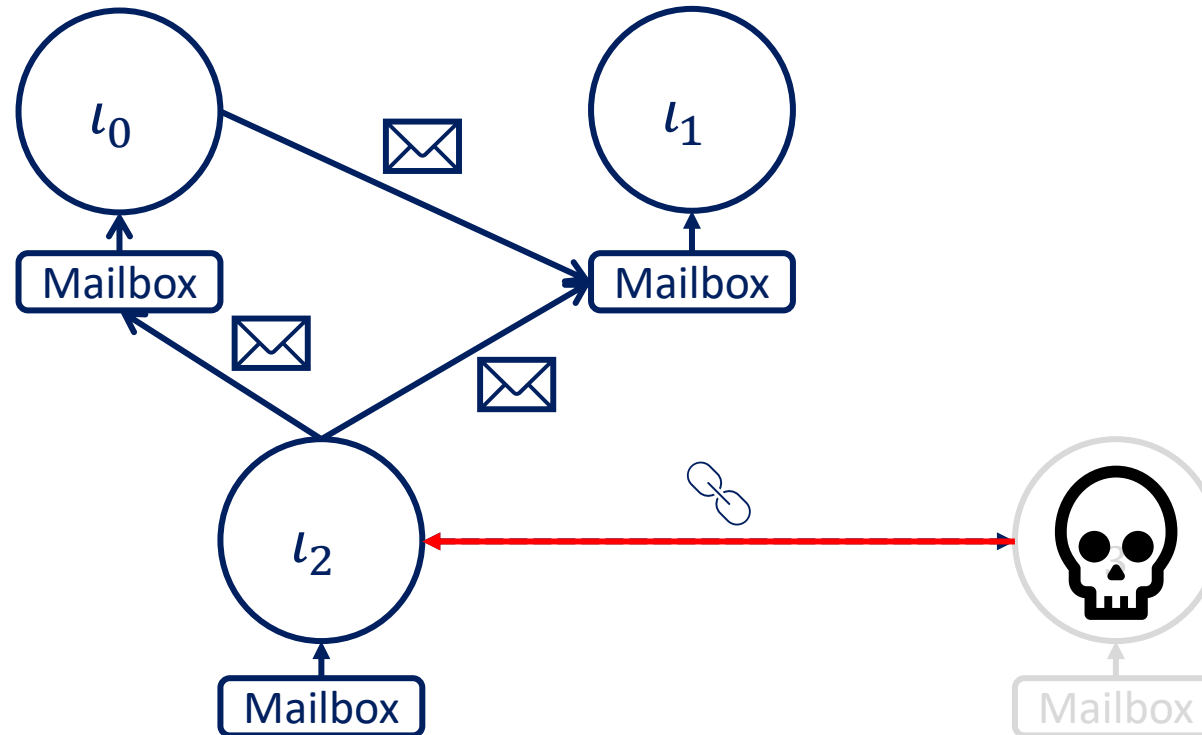
Modular semantics

Level of the semantics

Formal relation

Sequential semantics

$\langle K, e \rangle \rightarrow \langle K', e' \rangle$



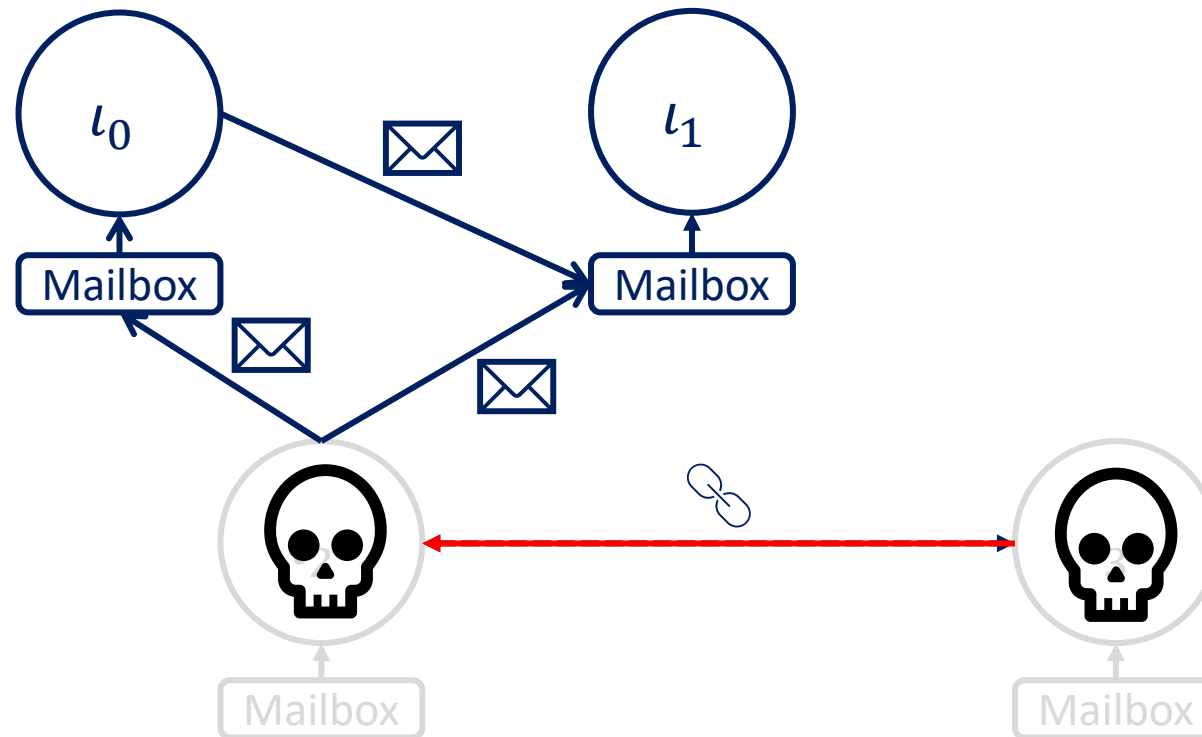
Modular semantics

Level of the semantics

Formal relation

Sequential semantics

$\langle K, e \rangle \rightarrow \langle K', e' \rangle$



Modular semantics

Level of the semantics

Formal relation

Sequential semantics

$$\langle K, e \rangle \rightarrow \langle K', e' \rangle$$

Process-local semantics

$$(K, e, mb, links, flag) \xrightarrow{a} (K', e', mb', links', flag')$$

Modular semantics

Level of the semantics

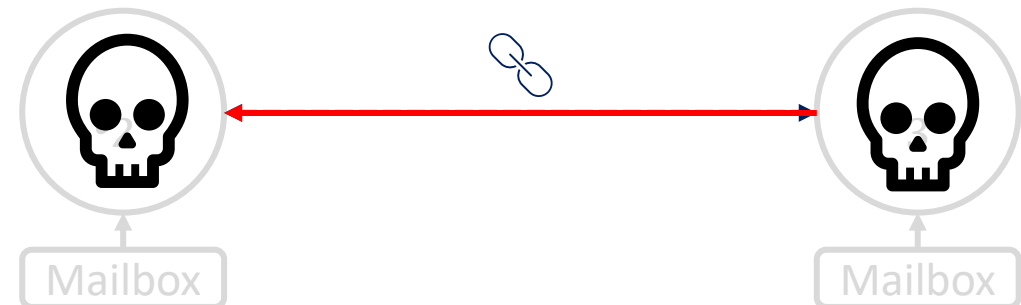
Formal relation

Sequential semantics

$$\langle K, e \rangle \rightarrow \langle K', e' \rangle$$

Process-local semantics

$$(K, e, mb, links, flag) \xrightarrow{a} (K', e', mb', links', flag')$$



Modular semantics

Level of the semantics

Formal relation

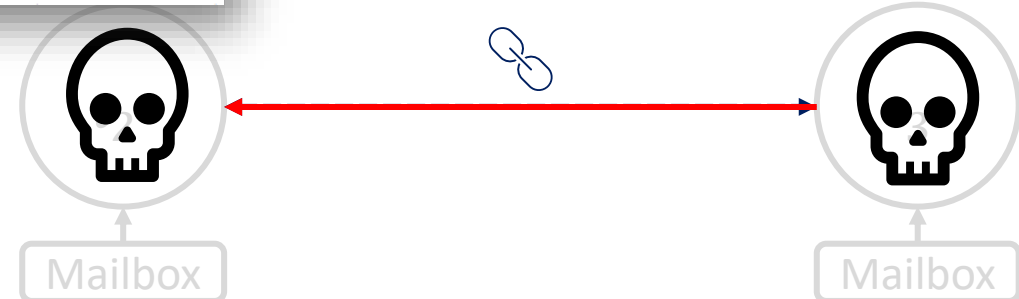
Sequential semantics

$$\langle K, e \rangle \rightarrow \langle K', e' \rangle$$

Process-local semantics

$$(K, e, mb, links, flag) \xrightarrow{a} (K', e', mb', links', flag')$$

$$\frac{\begin{array}{l} (v = \text{'kill'} \wedge b^e = \text{false} \wedge v' = \text{'killed'}) \vee \\ (b = \text{false} \wedge v = \text{'normal'} = v' \wedge \iota^s = \iota^d) \vee \\ (b = \text{false} \wedge v \neq \text{'normal'} \wedge v' = v \wedge (b^e = \text{true} \rightarrow \iota^s \in L) \wedge (b^e = \text{false} \rightarrow v \neq \text{'kill'})) \end{array}}{(K, r, q, L, b) \xrightarrow{arr(\iota^s, \iota^d, exit(v, b^e))} (\lambda \iota \Rightarrow (\iota, v')) <\$> L}$$



Modular semantics

Level of the semantics

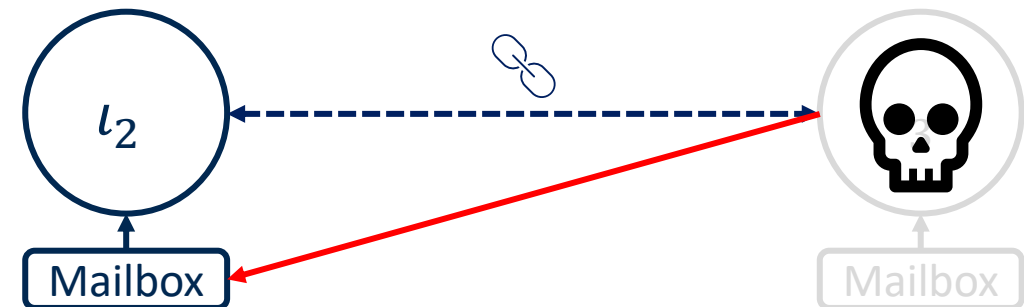
Formal relation

Sequential semantics

$$\langle K, e \rangle \rightarrow \langle K', e' \rangle$$

Process-local semantics

$$(K, e, mb, links, flag) \xrightarrow{a} (K', e', mb', links', flag')$$



Modular semantics

Level of the semantics

Formal relation

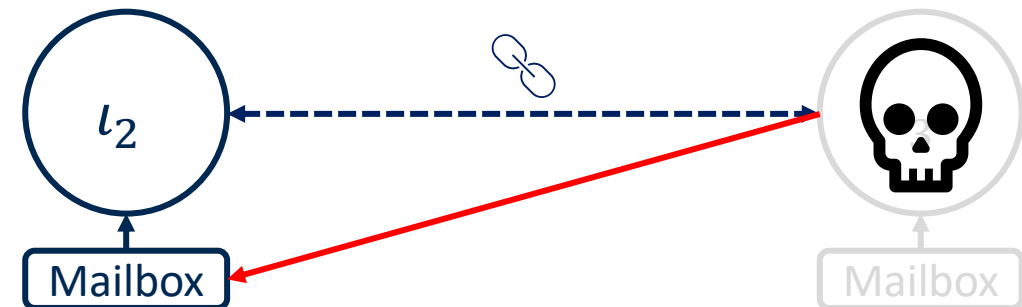
Sequential semantics

$$\langle K, e \rangle \rightarrow \langle K', e' \rangle$$

Process-local semantics

$$(K, e, mb, links, flag) \xrightarrow{a} (K', e', mb', links', flag')$$

$$\frac{b = true \wedge ((b^e = false \wedge v \neq 'kill') \vee (b^e = true \wedge \iota^s \in L))}{(K, r, q, L, b) \xrightarrow{arr(\iota^s, \iota^d, exit(v, b^e))} (K, r, push(q, \{'EXIT', \iota^s, v\}), L, b)}$$



Modular semantics

Level of the semantics	Formal relation
Sequential semantics	$\langle K, e \rangle \rightarrow \langle K', e' \rangle$
Process-local semantics	$(K, e, mb, links, flag) \xrightarrow{a} (K', e', mb', links', flag')$
Inter-process semantics	$(\mathcal{E}, \iota : p \parallel \Pi) \xrightarrow[\mathcal{O}]{\iota : a} (\mathcal{E}', \iota : p' \parallel \Pi)$

Modular semantics

Level of the semantics

Formal relation

Sequential semantics

$$\langle K, e \rangle \rightarrow \langle K', e' \rangle$$

Process-local semantics

$$(K, e, mb, links, flag) \xrightarrow{a} (K', e', mb', links', flag')$$

Inter-process semantics

$$(\mathcal{E}, \iota : p \parallel \Pi) \xrightarrow[\mathcal{O}]{\iota : a} (\mathcal{E}', \iota : p' \parallel \Pi)$$

$$p \xrightarrow{send(\iota^s, \iota^d, s)} p'$$

$$(\mathcal{E}, \iota^s : p \parallel \Pi) \xrightarrow[\mathcal{O}]{\iota^s : send(\iota^s, \iota^d, s)} (\mathcal{E}[(\iota^s, \iota^d) \mapsto^+ s], \iota^s : p' \parallel \Pi)$$

Modular semantics

Level of the semantics

Formal relation

Sequential semantics

$$\langle K, e \rangle \rightarrow \langle K', e' \rangle$$

Process-local semantics

$$(K, e, mb, links, flag) \xrightarrow{a} (K', e', mb', links', flag')$$

Inter-process semantics

$$(\mathcal{E}, \iota : p \parallel \Pi) \xrightarrow[\mathcal{O}]{\iota : a} (\mathcal{E}', \iota : p' \parallel \Pi)$$

$$p \xrightarrow{send(\iota^s, \iota^d, s)} p'$$

$$(\mathcal{E}, \iota^s : p \parallel \Pi) \xrightarrow[\mathcal{O}]{\iota^s : send(\iota^s, \iota^d, s)} (\mathcal{E}[(\iota^s, \iota^d) \mapsto^+ s], \iota^d)$$

$$\frac{p \xrightarrow{spawn(\iota_2, v^f, v^l, b)} p' \quad \begin{array}{l} L = \text{if } b \text{ then } \{\iota_1\} \text{ else } \emptyset \\ r = \text{result_of}(v^f, v_1, \dots, v_k) \end{array} \quad \iota_2 \notin \mathcal{O} \cup PIDs(\mathcal{E}) \cup PIDs(\iota_1 : p \parallel \Pi)}{(\mathcal{E}, \iota_1 : p \parallel \Pi) \xrightarrow[\mathcal{O}]{\iota_1 : spawn(\iota_2, v^f, [v_1, \dots, v_k], b)} (\mathcal{E}, \iota_2 : (\varepsilon, r, [\blacktriangleright], L, false) \parallel \iota_1 : p' \parallel \Pi)}$$

Program equivalence

$$C[e] \xrightarrow{\text{transformation}} C[e']$$

Program equivalence

$$C[e] \xrightarrow{\text{transformation}} C[e']$$

is a refactoring if

$$e \approx e'$$

Program equivalence

$$C[e] \xrightarrow{\text{transformation}} C[e']$$

is a refactoring if

$$e \approx e'$$

$$e_1 \leq_{\text{ctx}}^{\Gamma} e_2 := \\ \forall (C: \text{Context}): \langle \varepsilon, C[e_1] \rangle \Downarrow \Rightarrow \langle \varepsilon, C[e_2] \rangle \Downarrow$$

Program equivalence

$$e_1 \leq_{ctx}^{\Gamma} e_2 := \\ \forall (C: Context): \langle \varepsilon, C[e_1] \rangle \Downarrow \Rightarrow \langle \varepsilon, C[e_2] \rangle \Downarrow$$

Program equivalence

$$e_1 \leq_{ctx}^{\Gamma} e_2 := \\ \forall (C: Context): \langle \varepsilon, C[e_1] \rangle \Downarrow \Rightarrow \langle \varepsilon, C[e_2] \rangle \Downarrow$$

$$(e_1, e_2) \in \mathbb{E}_n, (K_1, K_2) \in \mathbb{K}_n, (v_1, v_2) \in \mathbb{V}_n, (exc_1, exc_2) \in \mathbb{X}_n, (r_1, r_2) \in \mathbb{R}_n \\ (e_1, e_2) \in \mathbb{E}^{\Gamma}, (K_1, K_2) \in \mathbb{K}^{\Gamma}, (v_1, v_2) \in \mathbb{V}^{\Gamma}, (exc_1, exc_2) \in \mathbb{X}^{\Gamma}, (r_1, r_2) \in \mathbb{R}^{\Gamma} \\ (\sigma_1, \sigma_2) \in \mathbb{G}_n^{\Gamma}$$

Program equivalence

$$e_1 \leq_{ctx}^{\Gamma} e_2 := \\ \forall (C: Context): \langle \varepsilon, C[e_1] \rangle \Downarrow \Rightarrow \langle \varepsilon, C[e_2] \rangle \Downarrow$$

$$(e_1, e_2) \in \mathbb{E}_n, (K_1, K_2) \in \mathbb{K}_n, (v_1, v_2) \in \mathbb{V}_n, (exc_1, exc_2) \in \mathbb{X}_n, (r_1, r_2) \in \mathbb{R}_n \\ (e_1, e_2) \in \mathbb{E}^{\Gamma}, (K_1, K_2) \in \mathbb{K}^{\Gamma}, (v_1, v_2) \in \mathbb{V}^{\Gamma}, (exc_1, exc_2) \in \mathbb{X}^{\Gamma}, (r_1, r_2) \in \mathbb{R}^{\Gamma} \\ (\sigma_1, \sigma_2) \in \mathbb{G}_n^{\Gamma}$$

$$r_1 \leq_{ciu} r_2 := \forall (K: FrameStack), \\ \langle K, r_1 \rangle \Downarrow \Rightarrow \langle K, r_2 \rangle \Downarrow$$

$$r_1 \leq_{ciu}^{\Gamma} r_2 := \forall \sigma, r_1[\sigma] \leq_{ciu} r_2[\sigma]$$

Program equivalence

$$e_1 \leq_{ctx}^{\Gamma} e_2 := \\ \forall (C: Context): \langle \varepsilon, C[e_1] \rangle \Downarrow \Rightarrow \langle \varepsilon, C[e_2] \rangle \Downarrow$$

$$(e_1, e_2) \in \mathbb{E}_n, (K_1, K_2) \in \mathbb{K}_n, (v_1, v_2) \in \mathbb{V}_n, (exc_1, exc_2) \in \mathbb{X}_n, (r_1, r_2) \in \mathbb{R}_n \\ (e_1, e_2) \in \mathbb{E}^{\Gamma}, (K_1, K_2) \in \mathbb{K}^{\Gamma}, (v_1, v_2) \in \mathbb{V}^{\Gamma}, (exc_1, exc_2) \in \mathbb{X}^{\Gamma}, (r_1, r_2) \in \mathbb{R}^{\Gamma} \\ (\sigma_1, \sigma_2) \in \mathbb{G}_n^{\Gamma}$$

$$r_1 \leq_{ciu} r_2 := \forall (K: FrameStack), \\ \langle K, r_1 \rangle \Downarrow \Rightarrow \langle K, r_2 \rangle \Downarrow$$

$$r_1 \leq_{ciu}^{\Gamma} r_2 := \forall \sigma, r_1[\sigma] \leq_{ciu} r_2[\sigma]$$

$$(e_1, e_2) \in \mathbb{E}^{\Gamma} \Leftrightarrow e_1 \leq_{ciu}^{\Gamma} e_2 \Leftrightarrow e_1 \leq_{ctx}^{\Gamma} e_2$$

Program equivalence and refactorings

Program equivalence and refactorings

$f(x)$ **when**
 $\text{length}(x) == 0 \rightarrow e_1;$
 $f(x) \rightarrow e_2.$

transformation

$f([]) \rightarrow e_1;$
 $f(x) \rightarrow e_2.$

Program equivalence and refactorings

$f(x)$ **when**
 $\text{length}(x) == 0 \rightarrow e_1;$
 $f(x) \rightarrow e_2.$

transformation

$f([]) \rightarrow e_1;$
 $f(x) \rightarrow e_2.$

is a refactoring if

$\forall f, x, e_i:$

$f(x)$ **when**
 $\text{length}(x) == 0 \rightarrow e_1;$
 $f(x) \rightarrow e_2.$

$\approx$

$f([]) \rightarrow e_1;$
 $f(x) \rightarrow e_2.$

if $x \notin \text{vars}(e_1)$

Program equivalence and refactorings

$f(x)$ **when**
 $\text{length}(x) == 0 \rightarrow e_1;$
 $f(x) \rightarrow e_2.$

transformation

$f([]) \rightarrow e_1;$
 $f(x) \rightarrow e_2.$

is a refactoring if

$\forall f, x, e_i:$

$f(x)$ **when**
 $\text{length}(x) == 0 \rightarrow e_1;$
 $f(x) \rightarrow e_2.$

$\approx$

$f([]) \rightarrow e_1;$
 $f(x) \rightarrow e_2.$

if $x \notin \text{vars}(e_1)$

Standard translation to Core Erlang + equivalence in Core Erlang

Program equivalence and refactorings

```
 $f(x)$  when  
   $\text{length}(x) == 0 \rightarrow e_1;$   
 $f(x) \rightarrow e_2.$ 
```

```
 $f([]) \rightarrow e_1;$   
 $f(x) \rightarrow e_2.$ 
```

Program equivalence and refactorings

```
f(x) when  
  length(x) == 0 -> e1;  
f(x) -> e2.
```



```
f([]) -> e1;  
f(x) -> e2.
```



Program equivalence and refactorings

```
f(x) when  
  length(x) == 0 -> e1;  
f(x) -> e2.
```



```
f/1 = fun (_0) ->  
  case (_0) of  
    <x> when try let <_1> =  
      call 'erlang':'length'(x)  
      in call 'erlang':'=='(_1, 0)  
    of <Try> -> Try  
    catch <T,R> -> 'false'  
  -> e1  
  <_3> when 'true' -> e2  
  <_2> when 'true' ->  
    primop 'match_fail'({'function_clause',_2})  
end
```

```
f([]) -> e1;  
f(x) -> e2.
```



```
f/1 = fun (_0) ->  
  case (_0) of  
    <[]> when 'true' -> e1  
    <x> when 'true' -> e2  
    <_1> when 'true' ->  
      primop 'match_fail'({'function_clause',_1})  
  end
```

Program equivalence and refactorings

```
f(x) when  
  length(x) == 0 -> e1;  
f(x) -> e2.
```



```
f/1 = fun (_0) ->  
  case (_0) of  
    <x> when try let <_1> =  
      call 'erlang':'length'(x)  
      in call 'erlang':'=='(_1, 0)  
    of <Try> -> Try  
    catch <T,R> -> 'false'  
  -> e1  
  <_3> when 'true' -> e2  
  <_2> when 'true' ->  
    primop 'match_fail'({'function_clause',_2})  
end
```

≈

```
f([]) -> e1;  
f(x) -> e2.
```



```
f/1 = fun (_0) ->  
  case (_0) of  
    <[]> when 'true' -> e1  
    <x> when 'true' -> e2  
    <_1> when 'true' ->  
      primop 'match_fail'({'function_clause',_1})  
  end
```

Formal semantics and proofs...

Formal semantics and proofs...



... on paper

While a paper-based semantics is precise, it **is manual**. Proofs are written and checked manually; thus, **errors can easily slip through**.

Formal semantics and proofs...



... on paper

While a paper-based semantics is precise, it **is manual**. Proofs are written and checked manually; thus, **errors can** easily **slip through**.



... in a proof assistant

Formalising a semantics in a proof assistant is a major effort. But, **proofs are machine-checked**, there is **no** room for **imprecision**, while **proving is interactive**.

 So, goal achieved, right?

 So, goal achieved, right?

Sadly, people do not seem to be that interested in refactorings.

 So, goal achieved, right?

Sadly, people do not seem to be that interested in refactorings.

 What else can we do with the semantics?

Program equivalence – constant folding

remove_file	:	0.000 s	4.0 kB
parse_module	:	0.001 s	6.4 kB
transform_module	:	0.000 s	6.4 kB
lint_module	:	0.003 s	6.7 kB
compile_directives	:	0.000 s	6.7 kB
expand_records	:	0.000 s	6.7 kB
core	:	0.000 s	28.4 kB
sys_core_fold	:	0.001 s	16.9 kB
sys_core_alias	:	0.000 s	16.9 kB
core_transforms	:	0.000 s	16.9 kB
sys_core_bsm	:	0.000 s	16.9 kB
v3_kernel	:	0.000 s	16.4 kB
beam_kernel_to_ssa	:	0.000 s	11.2 kB
beam_ssa_bool	:	0.000 s	11.2 kB
beam_ssa_share	:	0.000 s	11.2 kB
beam_ssa_recv	:	0.000 s	11.2 kB

Program equivalence – constant folding

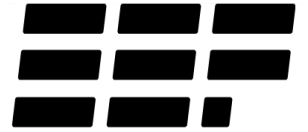
remove_file	:	0.000	s	4.0	kB
parse_module	:	0.001	s	6.4	kB
transform_module	:	0.000	s	6.4	kB
lint_module	:	0.003	s	6.7	kB
compile_directives	:	0.000	s	6.7	kB
expand_records	:	0.000	s	6.7	kB
core	:	0.000	s	28.4	kB
sys_core_fold	:	0.001	s	16.9	kB
sys_core_alias	:	0.000	s	16.9	kB
core_transforms	:	0.000	s	16.9	kB
sys_core_bsm	:	0.000	s	16.9	kB
v3_kernel	:	0.000	s	16.4	kB
beam_kernel_to_ssa	:	0.000	s	11.2	kB
beam_ssa_bool	:	0.000	s	11.2	kB
beam_ssa_share	:	0.000	s	11.2	kB
beam_ssa_recv	:	0.000	s	11.2	kB

```
255     expr(#c_seq{arg=Arg0,body=B0}=Seq0, Ctxt, Sub) ->
256         %% Optimise away pure literal arg as its value is ignored.
257         B1 = body(B0, Ctxt, Sub),
258         Arg = body(Arg0, effect, Sub),
259         case will_fail(Arg) of
260             true ->
261                 Arg;
262             false ->
263                 %% Arg cannot be "values" here - only a single value
264                 %% make sense here.
265                 case {Ctxt,is_safe_simple(Arg)} of
266                     {effect,true} -> B1;
267                     {effect,false} ->
268                         case is_safe_simple(B1) of
269                             true -> Arg;
270                             false -> Seq0#c_seq{arg=Arg,body=B1}
271                         end;
272                     {value,true} -> B1;
273                     {value,false} -> Seq0#c_seq{arg=Arg,body=B1}
274                 end
275         end;
```

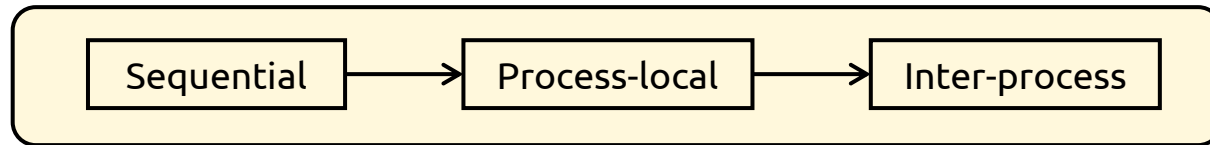
```
255     expr(#c_seq{arg=Arg0,body=B0}=Seq0, Ctxt, Sub) ->
256         %% Optimise away pure literal arg as its value is ignored.
257         B1 = body(B0, Ctxt, Sub),
258         Arg = body(Arg0, effect, Sub),
259         case will_fail(Arg) of
260             true ->
261                 Arg;
262             false ->
263                 %% Arg cannot be "values" here - only a single value
264                 %% make sense here.
265                 case {Ctxt,is_safe_simple(Arg)} of
266                     {effect,true} -> B1;
267                     {effect,false} ->
268                         case is_safe_simple(B1) of
269                             true -> Arg;
270                             false -> Seq0#c_seq{arg=Arg,body=B1}
271                         end;
272                     {value,true} -> B1;
273                     {value,false} -> Seq0#c_seq{arg=Arg,body=B1}
274                 end
275         end;
```

Not formalised yet

Reference interpreter



**Inductive
definitions**



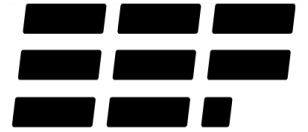
Languages:

Rocq

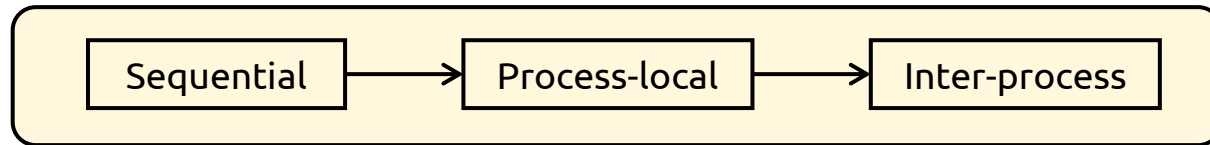
Erlang

Haskell

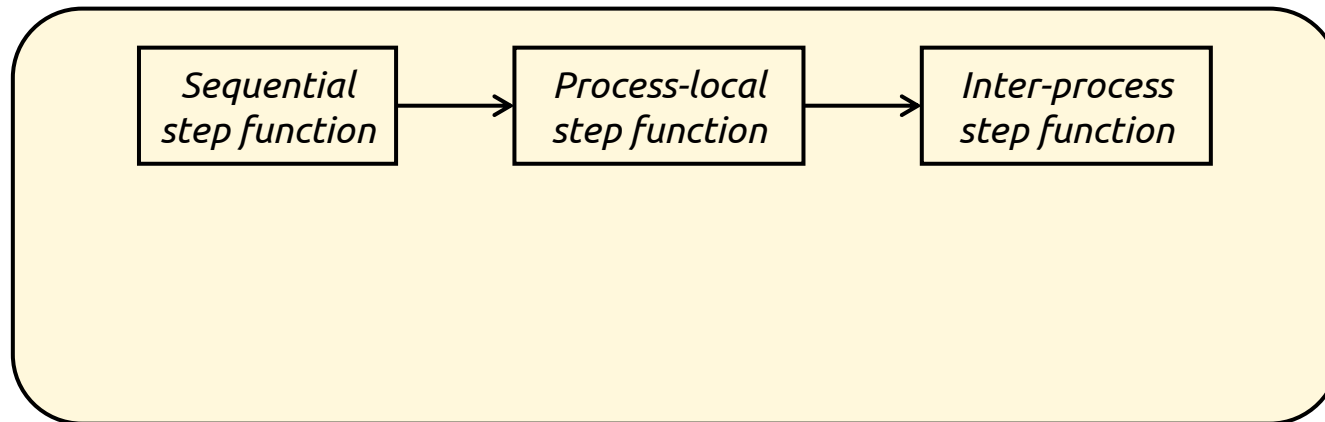
Reference interpreter



**Inductive
definitions**



**Function
definitions**



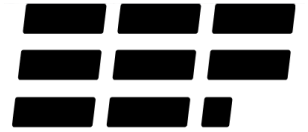
Languages:

Rocq

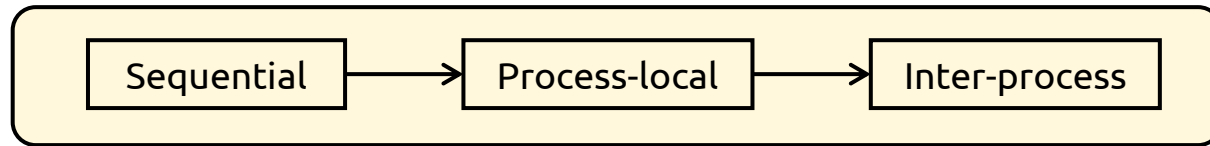
Erlang

Haskell

Reference interpreter



**Inductive
definitions**

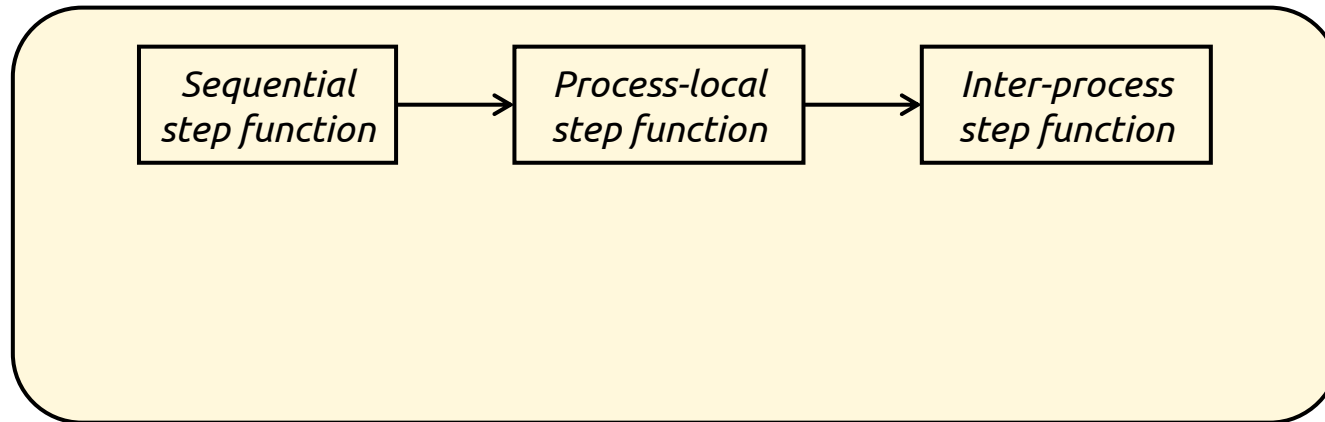


$\approx$

$\approx$

$\approx$

**Function
definitions**



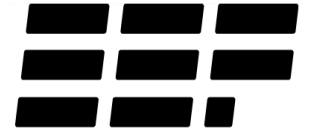
Languages:

Rocq

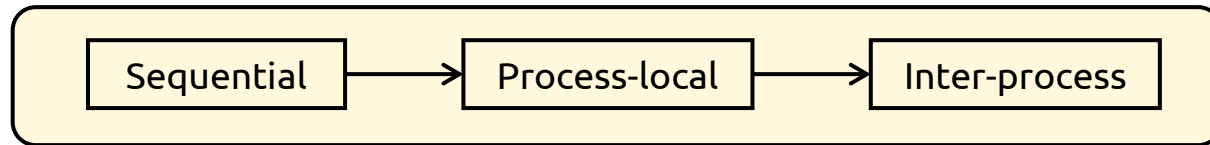
Erlang

Haskell

Reference interpreter



**Inductive
definitions**

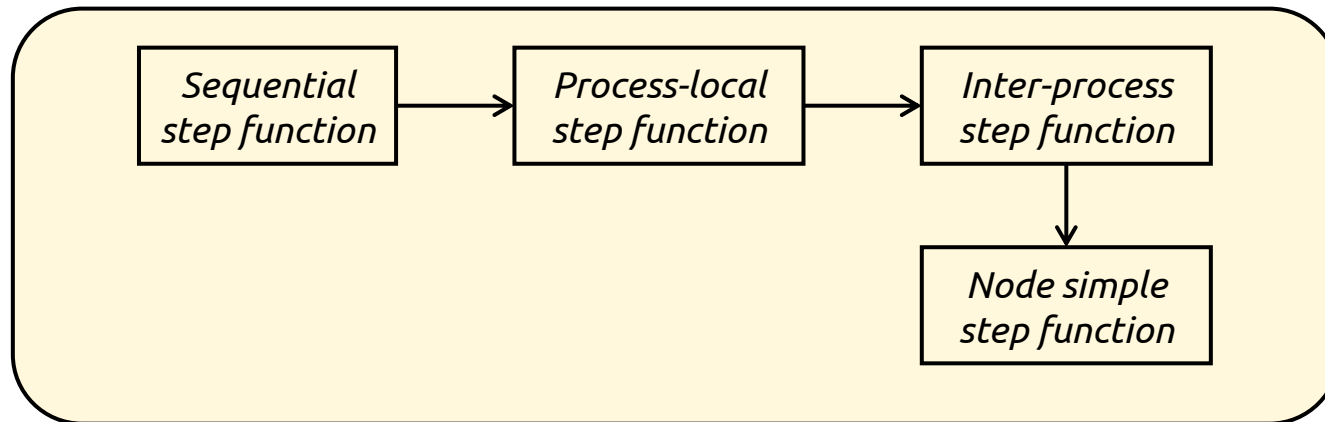


$\approx$

$\approx$

$\approx$

**Function
definitions**



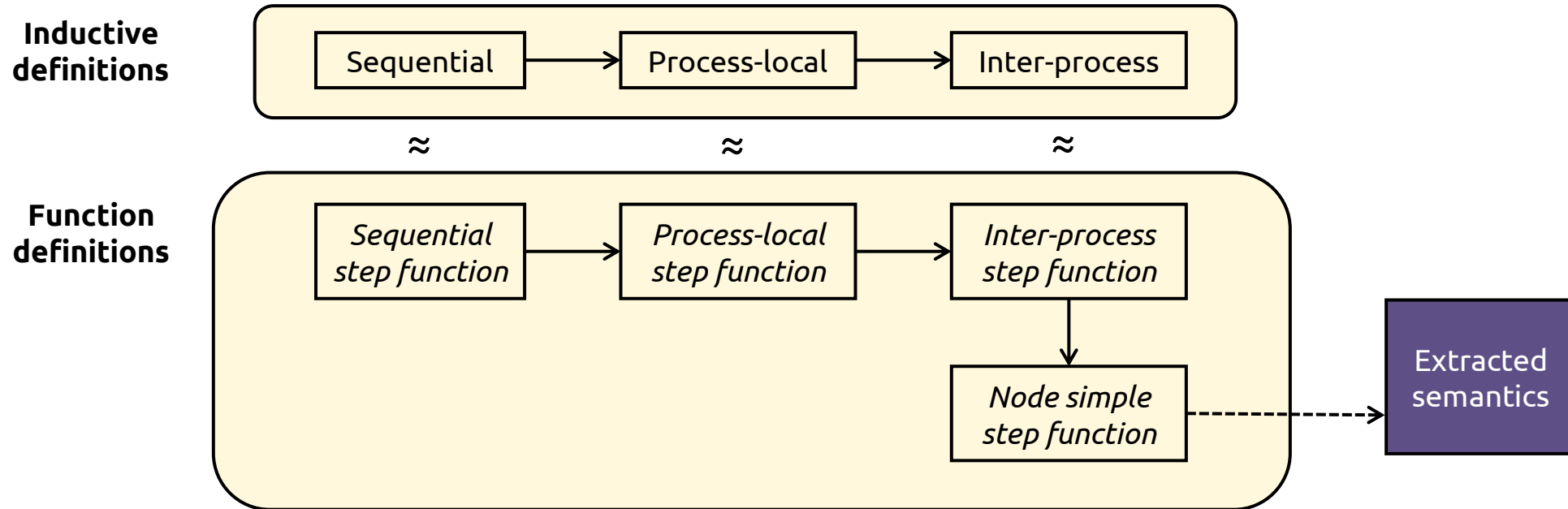
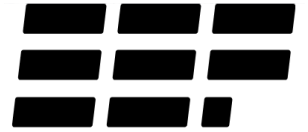
Languages:

Rocq

Erlang

Haskell

Reference interpreter



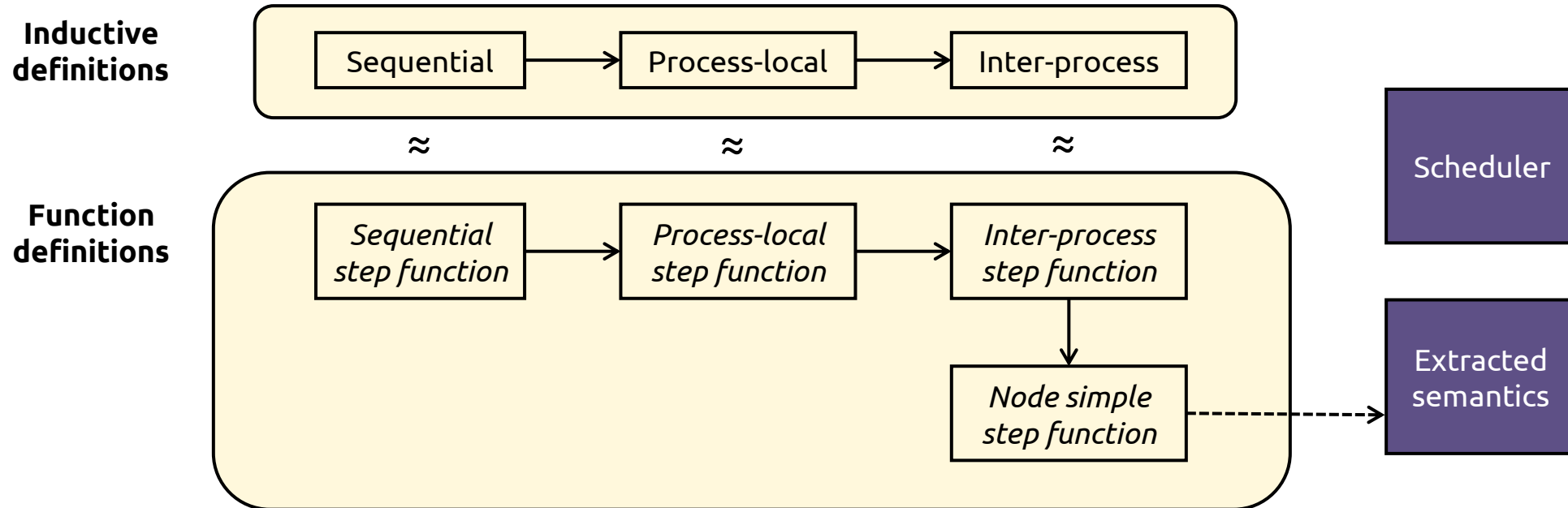
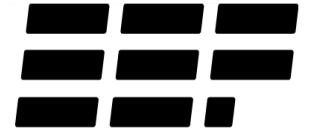
Languages:

Rocq

Erlang

Haskell

Reference interpreter



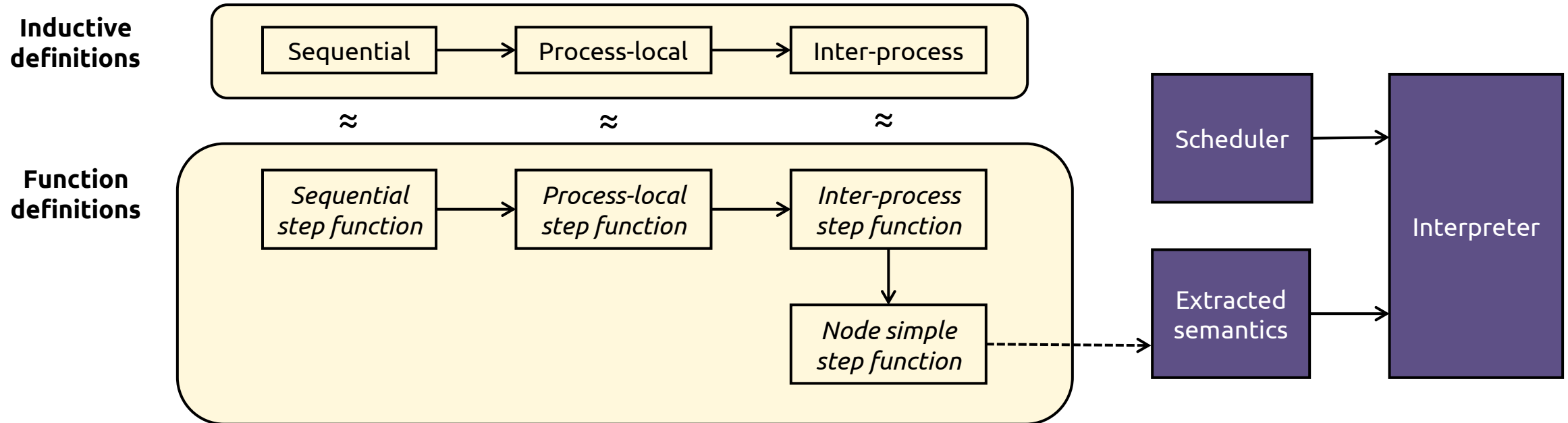
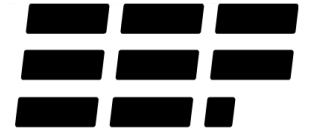
Languages:

Rocq

Erlang

Haskell

Reference interpreter



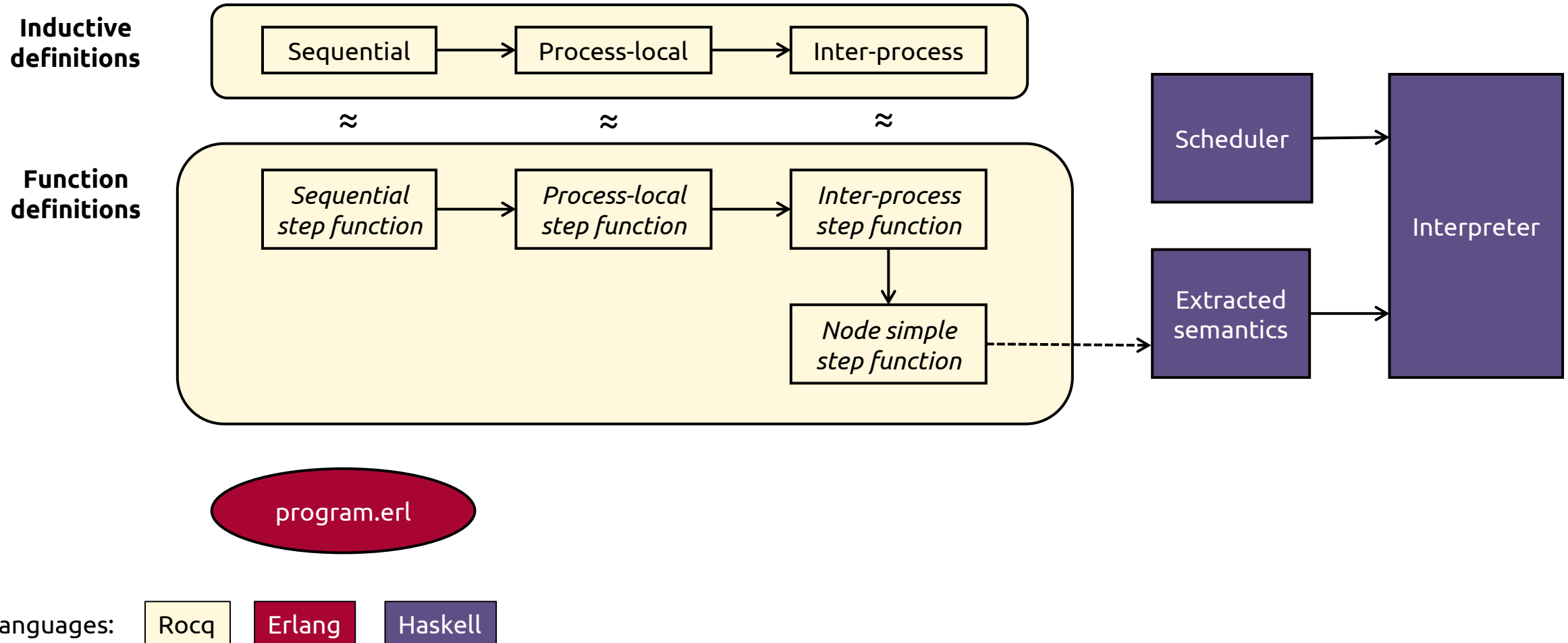
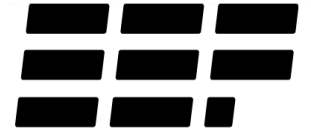
Languages:

Rocq

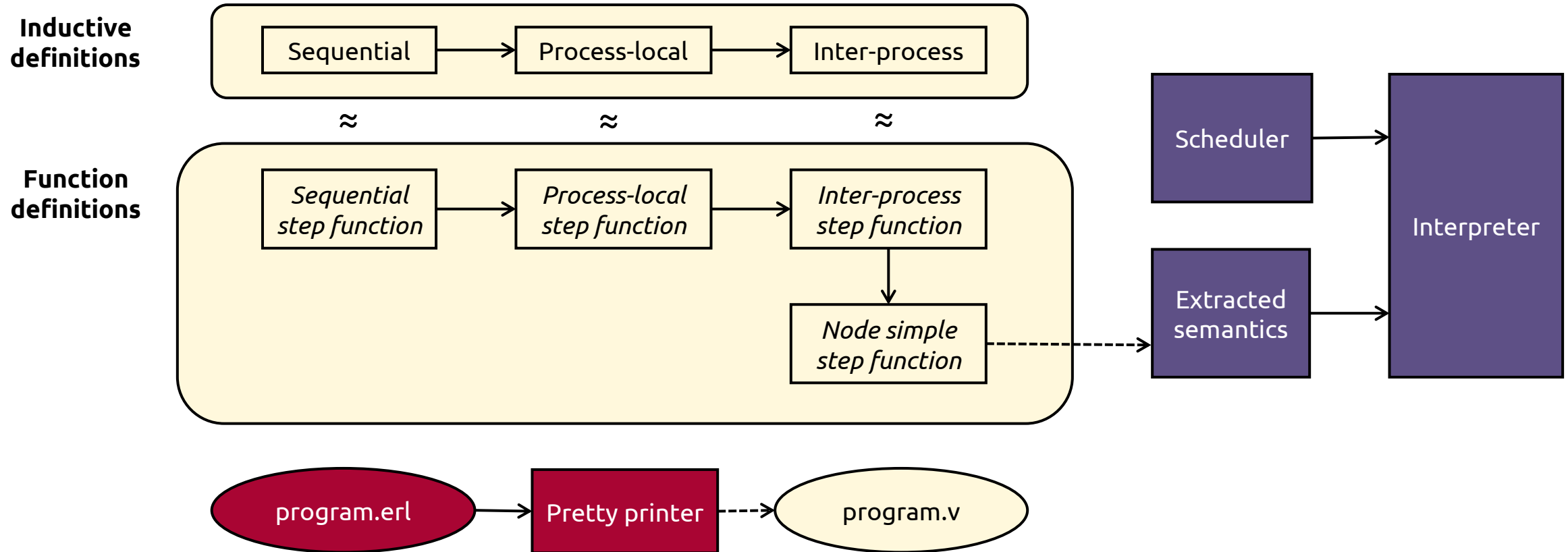
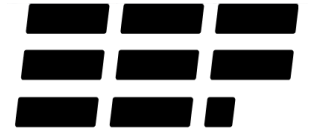
Erlang

Haskell

Reference interpreter



Reference interpreter



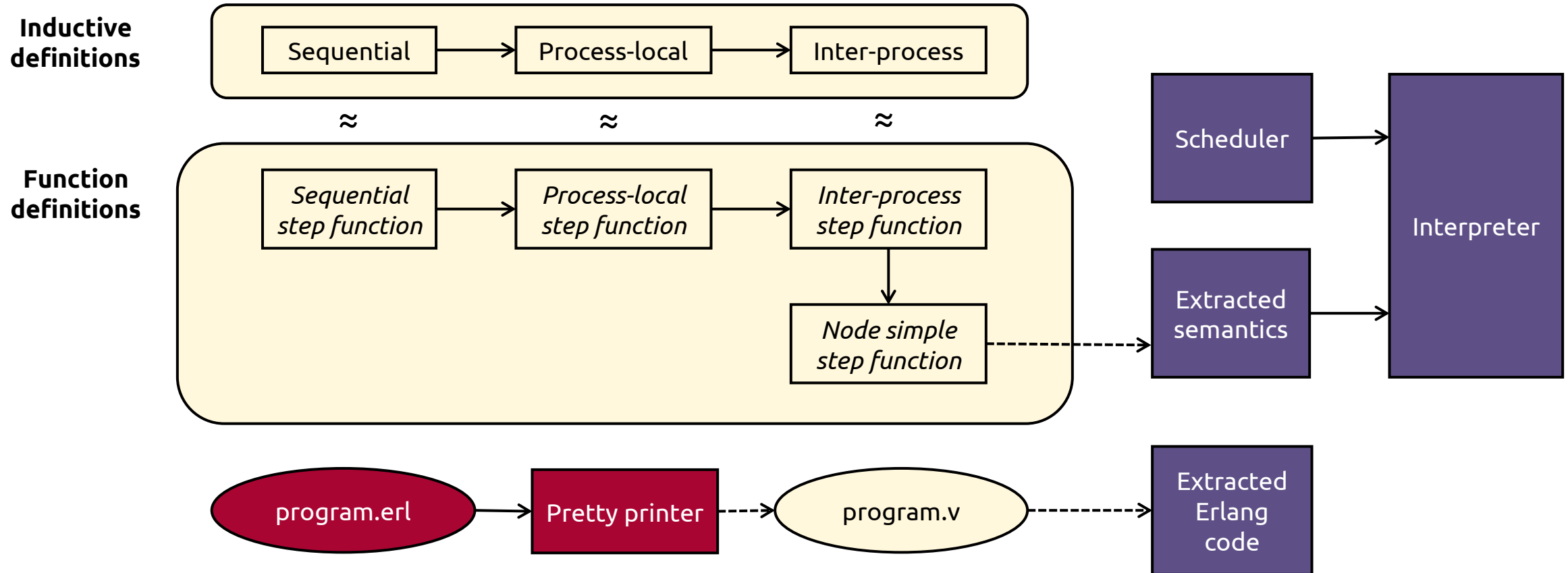
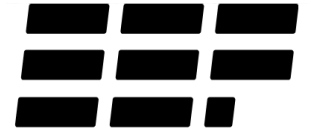
Languages:

Rocq

Erlang

Haskell

Reference interpreter



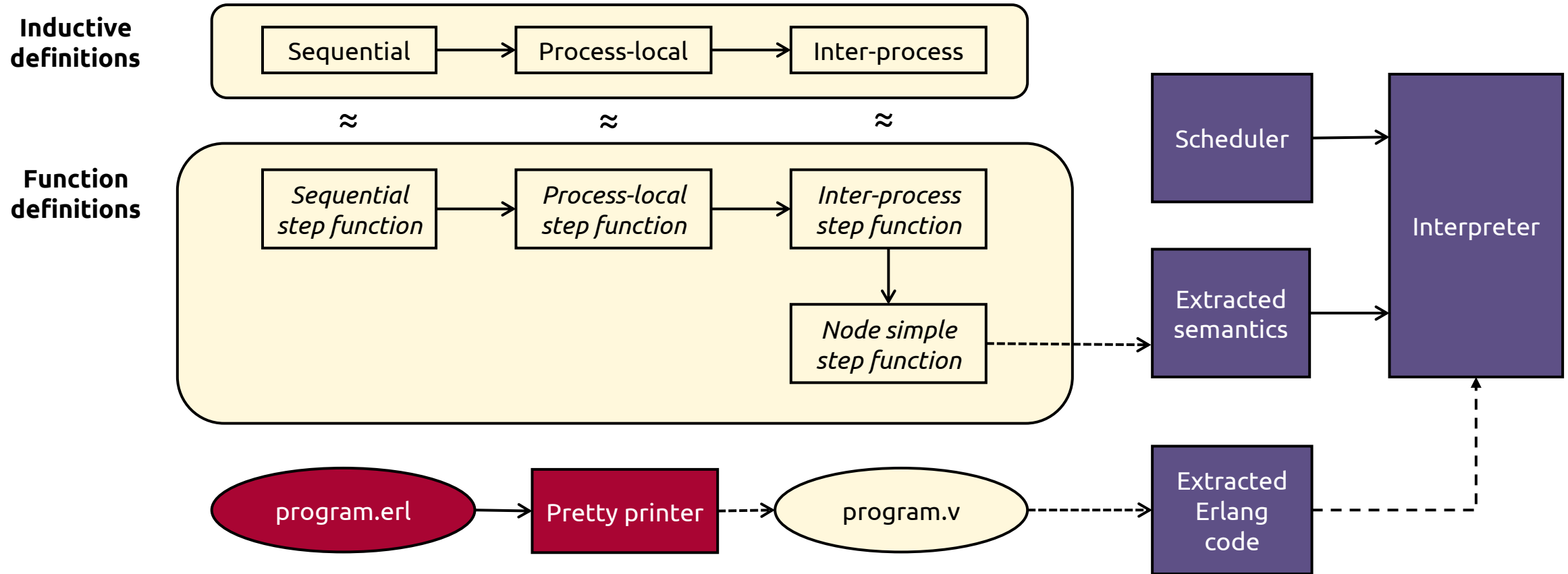
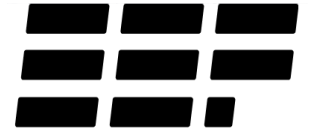
Languages:

Rocq

Erlang

Haskell

Reference interpreter



Languages:



		Execution time (%)				Memory usage (%)				Max mem.	runtime
		subst	map/set	sched	other	subst	map/set	sched	other	in 2 min.	(est.)
life (10x10 board, 10000 iters)	raw Coq extraction	33.6	40.8	5.8	19.8	38.5	49.5	5.0	7.0	984 MB	1.4 h
	with 1st solver	33.6	40.8	5.8	19.8	38.5	49.5	5.0	7.0	984 MB	1.4 h
	with 2nd solver	33.6	40.8	5.8	19.8	38.5	49.5	5.0	7.0	984 MB	1.4 h
	with 3rd solver	33.6	40.8	5.8	19.8	38.5	49.5	5.0	7.0	984 MB	1.4 h
	with 4th solver	33.6	40.8	5.8	19.8	38.5	49.5	5.0	7.0	984 MB	1.4 h
	with 5th solver	33.6	40.8	5.8	19.8	38.5	49.5	5.0	7.0	984 MB	1.4 h

Performance issues

		Execution time (%)				Memory usage (%)				Max mem. in 2 min.	runtime (est.)
		subst	map/set	sched	other	subst	map/set	sched	other		
life (10x10 board, 10000 iters)	raw Coq extraction	33.6	40.8	5.8	19.8	38.5	49.5	5.0	7.0	984 MB	1.4 h
	strict substitutions	92.1	6.1	0.5	1.3	90.5	8.1	0.7	0.7	84 MB	16.2 h

- Strictly evaluating substitutions

Performance issues

		Execution time (%)				Memory usage (%)				Max mem. in 2 min.	runtime (est.)
		subst	map/set	sched	other	subst	map/set	sched	other		
life (10x10 board, 10000 iters)	raw Coq extraction	33.6	40.8	5.8	19.8	38.5	49.5	5.0	7.0	984 MB	1.4 h
	strict substitutions	92.1	6.1	0.5	1.3	90.5	8.1	0.7	0.7	84 MB	16.2 h
	maps/sets replaced	97.3	1.2	0.5	1.0	97.5	1.0	0.7	0.8	83 MB	15.8 h

- Strictly evaluating substitutions
- Using strict data structures

Performance issues

		Execution time (%)				Memory usage (%)				Max mem. in 2 min.	runtime (est.)
		subst	map/set	sched	other	subst	map/set	sched	other		
life (10x10 board, 10000 iters)	raw Coq extraction	33.6	40.8	5.8	19.8	38.5	49.5	5.0	7.0	984 MB	1.4 h
	strict substitutions	92.1	6.1	0.5	1.3	90.5	8.1	0.7	0.7	84 MB	16.2 h
	maps/sets replaced	97.3	1.2	0.5	1.0	97.5	1.0	0.7	0.8	83 MB	15.8 h
	improved substitution	96.2	1.1	0.9	1.8	85.2	4.3	4.8	5.7	64 MB	6.6 h

- Strictly evaluating substitutions
- Using strict data structures
- Simplifying substitution

Performance issues

		Execution time (%)				Memory usage (%)				Max mem. in 2 min.	runtime (est.)
		subst	map/set	sched	other	subst	map/set	sched	other		
life (10x10 board, 10000 iters)	raw Coq extraction	33.6	40.8	5.8	19.8	38.5	49.5	5.0	7.0	984 MB	1.4 h
	strict substitutions	92.1	6.1	0.5	1.3	90.5	8.1	0.7	0.7	84 MB	16.2 h
	maps/sets replaced	97.3	1.2	0.5	1.0	97.5	1.0	0.7	0.8	83 MB	15.8 h
	improved substitution	96.2	1.1	0.9	1.8	85.2	4.3	4.8	5.7	64 MB	6.6 h

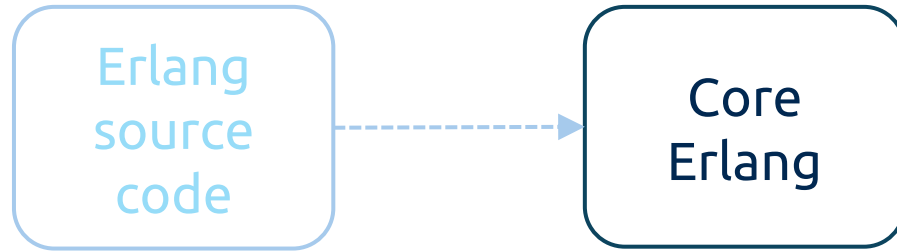
- Strictly evaluating substitutions
- Using strict data structures
- Simplifying substitution

Further possible improvements:

- Using environments instead of substitutions

A verified, optimising compiler for Erlang

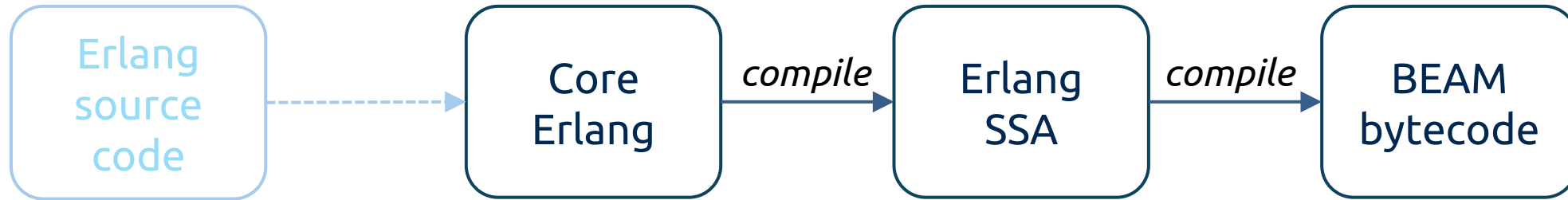
A verified, optimising compiler for Erlang



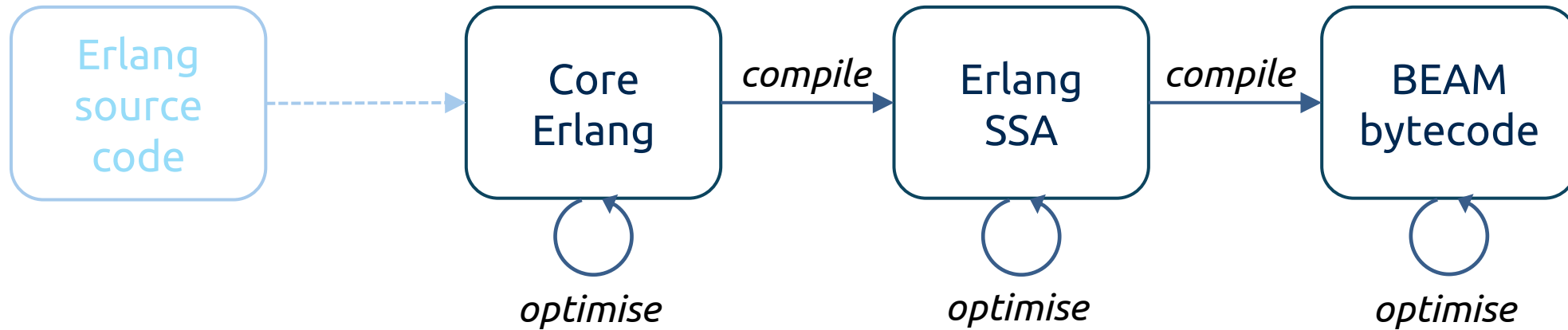
A verified, optimising compiler for Erlang



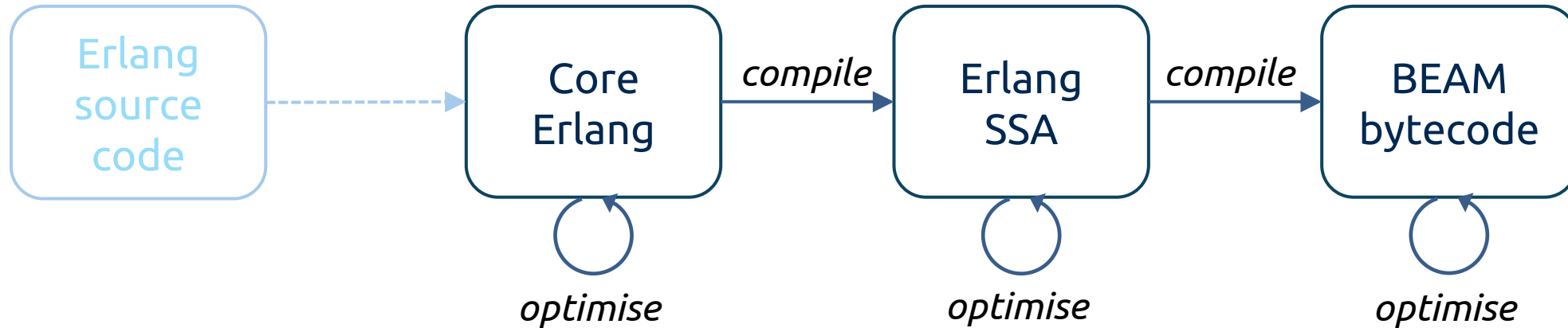
A verified, optimising compiler for Erlang



A verified, optimising compiler for Erlang

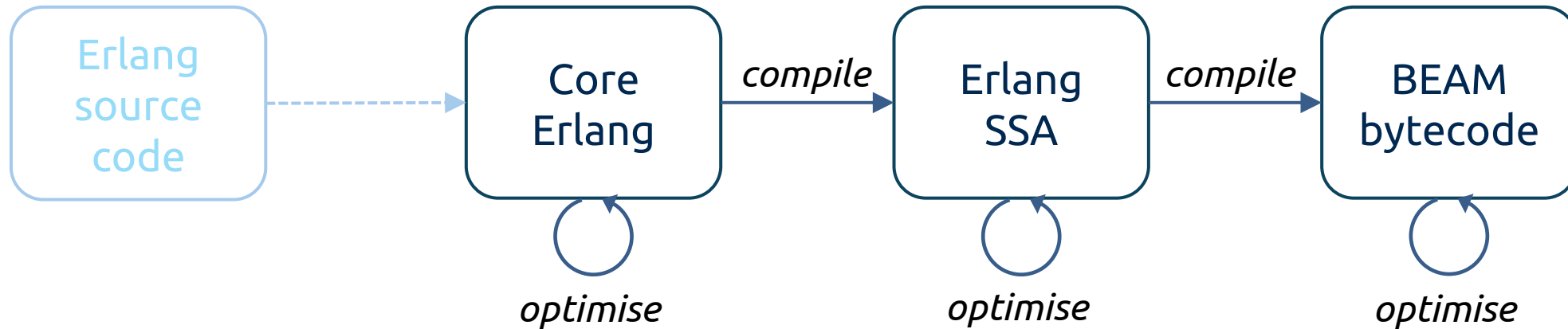


A verified, optimising compiler for Erlang



$$\begin{aligned} \forall C, D: \text{compile}(C) = D \rightarrow C \approx D \\ \forall C, D: \text{optimise}(C) = D \rightarrow C \approx D \end{aligned}$$

A verified, optimising compiler for Erlang



$$\begin{aligned} \forall C, D: \text{compile}(C) = D \rightarrow C \approx D \\ \forall C, D: \text{optimise}(C) = D \rightarrow C \approx D \end{aligned}$$



NKKP STARTING



D. Horpácsi, P. Bereczky, D. Lukács, G. Turán, S. Thompson

Conclusions

Conclusions

We have an open source formal semantics of Core Erlang in a proof assistant

Conclusions

We have an open source formal semantics of Core Erlang in a proof assistant

Formal semantics is the infrastructure for trustworthy tools

Conclusions

We have an open source formal semantics of Core Erlang in a proof assistant

Formal semantics is the infrastructure for trustworthy tools

Reference interpreter

Static analysis

**Equivalence
verification**

Conclusions

We have an open source formal semantics of Core Erlang in a proof assistant

Formal semantics is the infrastructure for trustworthy tools

Reference interpreter

Static analysis

**Equivalence
verification**

Further proven-correct refactorings and optimisations

Conclusions

We have an open source formal semantics of Core Erlang in a proof assistant

Formal semantics is the infrastructure for trustworthy tools

Reference interpreter

Static analysis

**Equivalence
verification**

Further proven-correct refactorings and optimisations

A verified compiler for Erlang

Thank you for your attention!

Contact

berpeti@inf.elte.hu

harp-project.github.io

